UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERIA

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS Y COMPUTACION

TEMA: Sistema de Gestión de Reservas de Ambientes Universitarios

CURSO: CONSTRUCCION DE SOFTWARE

DOCENTE: ING. VEGA FLORES JORGE ALBERTO

ALUMNO: ROJAS GUTIERREZ POOL FABIAN

CICLO: IX

HUANCAYO – 2026

RESUMEN EJECUTIVO **4**

1. SEMANA 01: FUNDAMENTOS, ACTORES Y ALCANCE DEL SISTEMA **5**
1.1. Situación problemática institucional 5
1.2. Actores del sistema y matriz de responsabilidades 6
1.3. Requerimientos funcionales atendidos 6
1.4. Alcance y límites del prototipo 7

2. SEMANA 02: CICLO DE VIDA DEL SOFTWARE **8**
2.1. Modelo seleccionado 8
2.2. Justificación técnica 9
2.3. Gestión de riesgos de la iteración 10

3. SEMANA 03: ESTÁNDARES DE CONSTRUCCIÓN DE SOFTWARE **11**
3.1. Estándares de nomenclatura 11
3.2. Estándares de arquitectura y organización de carpetas 11
3.3. Estándares de versionado y commits 12
3.4. Estándares de interfaz y accesibilidad 13

4. SEMANA 04: ARQUITECTURA Y MODELO DE DATOS **14**
4.1. Arquitectura de la solución 14
4.2. Modelo entidad-relación 16
4.3. Diccionario de datos 17
4.4. Integridad, índices y reglas en la base de datos 18
4.5. Evidencia de la base de datos implementada 19
4.6. Seguridad de la persistencia y de la autenticación 19

5. DISEÑO PREVIO DE LA INTERFAZ Y WIREFRAMES **20**
5.1. Enfoque de diseño centrado en el usuario 20
5.2. Wireframe de la pantalla de acceso 21
5.3. Wireframe del Inicio 22
5.4. Wireframe del catálogo y gestión de ambientes 23
5.5. Wireframe de la gestión y el registro de usuarios 24
5.6. Wireframe del registro de reserva 26
5.7. Wireframe del listado de reservas con filtros 27
5.8. Wireframe de la vista de detalle y auditoría 28
5.9. Trazabilidad entre wireframe e implementación 29

6. SEMANA 05: MÓDULOS DE ENTRADA DE DATOS **29**
6.1. Formulario de acceso 30
6.2. Formulario de ambientes 32
6.3. Formulario de reservas 34
6.4. Formulario de usuarios 35
6.5. Campos, obligatoriedad y destino de los datos 37

7. SEMANA 06: VALIDACIÓN Y CAPTURA DE DATOS **38**
7.1. Estrategia de doble barrera 38
7.2. Implementación de las reglas de negocio RN-01 a RN-06 38
7.3. Validaciones de frontend y mensajes al usuario 39
7.4. Tratamiento uniforme de errores en el backend 40
7.5. Evidencia de las validaciones 41
7.6. Trazabilidad entre requerimientos y reglas 45

8. SEMANA 07: REUTILIZACIÓN DE CÓDIGO **45**
8.1. Estrategia de componentes 45
8.2. Catálogo de componentes mínimos 45
8.3. Evidencia de los componentes en uso 47
8.4. Beneficios observados 48

9. SEMANA 08: REFINAMIENTO DE CÓDIGO Y DEUDA TÉCNICA **49**

9.1. Refactorizaciones aplicadas 49
9.2. Mejoras de claridad y seguridad 49
9.3. Deuda técnica identificada 50

10. SEMANA 09: MÓDULOS DE SALIDA, PANEL DE INICIO Y AUDITORÍA 50
10.1. Indicadores del Panel de Inicio 50
10.2. Visualizaciones de tendencia y ocupación 53
10.3. Listado de reservas y vistas por rol 54
10.4. Detalle de la reserva y trazabilidad de auditoría 56
10.5. Consideraciones de rendimiento y consulta 57

11. RESPUESTA A LA PREGUNTA DE REFLEXIÓN FINAL 57
11.1. Estándares: previsibilidad y reducción de la carga cognitiva 58
11.2. Validación: confianza en el dato y prevención del error 58
11.3. Reutilización: coherencia y economía de esfuerzo 59
11.4. Refinamiento: reducción de deuda y capacidad de evolución 59
11.5. Conclusión 59

REFERENCIAS BIBLIOGRÁFICAS 60

ANEXO A. ESPECIFICACIÓN TÉCNICA DE LA API BACKEND (OPENAPI / SWAGGER) 61

RESUMEN EJECUTIVO

El presente informe documenta el proceso de análisis, diseño, construcción y refinamiento del sistema **ReservaUPLA**, una aplicación web para la gestión de reservas de ambientes universitarios de la Universidad Peruana Los Andes. El documento se organiza en once secciones que corresponden a las semanas 01 a 09 de la asignatura Construcción de Software y a la pregunta de reflexión final del caso práctico, e integra la evidencia obtenida directamente del sistema en ejecución.

La solución se implementó con una arquitectura limpia en cuatro capas sobre **ASP.NET Core 8** y **Entity Framework Core 8**, una interfaz **React 19.3.1** construida con **Vite 8.3.5** y **Tailwind CSS 3.4.3**, y una base de datos **PostgreSQL 16**. El estado verificado al cierre del informe comprende 3 entidades persistentes, 6 ambientes, 8 usuarios y 27 reservas, de las cuales 18 están aprobadas, 5 pendientes, 3 rechazadas y 1 cancelada. El ambiente con mayor ocupación es el Laboratorio de Cómputo 101, con 8 reservas aprobadas.

Los resultados se presentan con trazabilidad hacia los ocho requerimientos funcionales, las seis reglas de negocio, el catálogo de nueve componentes reutilizables y los cinco indicadores mínimos del tablero. Todas las capturas corresponden al sistema real en funcionamiento y fueron tomadas sobre la base de datos institucional descrita; no se emplearon prototipos simulados ni datos ficticios para sustentar las afirmaciones técnicas.

Palabras clave: construcción de software, arquitectura por capas, validación de datos, reutilización de componentes, usabilidad, trazabilidad, ReservaUPLA.

1. SEMANA 01: FUNDAMENTOS, ACTORES Y ALCANCE DEL SISTEMA

1.1. Situación problemática institucional

La Universidad Peruana Los Andes administra ambientes de distinta naturaleza —laboratorios de cómputo, laboratorios especializados, auditorios y salas de reuniones— cuya disponibilidad condiciona el desarrollo de sesiones de clase, evaluaciones y actividades institucionales. En el escenario descrito por el caso práctico, la asignación de estos espacios se realiza mediante mensajes, llamadas telefónicas o documentos físicos, es decir, a través de canales que no comparten un registro común ni un estado verificable en tiempo real.

Esa forma de operar produce cuatro efectos observables. Primero, **duplicidad de reservas**, porque dos solicitudes pueden comprometer el mismo ambiente en la misma fecha y franja horaria sin que ninguna instancia lo detecte. Segundo, **pérdida de solicitudes**, ya que el mensaje o la boleta física no constituyen un registro persistente y consultable. Tercero, **ausencia de trazabilidad**, pues no es posible determinar quién solicitó, quién aprobó y cuándo se tomó la decisión. Cuarto, **desconocimiento de la disponibilidad real**, que obliga al docente a trasladarse hasta el pabellón para comprobar si el aula está libre.

El análisis se sintetizó en la siguiente relación causa–efecto, en la que cada consecuencia se vincula con el requerimiento que la solución debe atender. Esta matriz permitió justificar el alcance y evitar funcionalidades que no respondieran a un problema real.

Tabla 1. Relación causa–efecto y requerimiento asociado

Causa identificada	Efecto sobre la operación	Requerimiento que la atiende
Registro disperso en mensajes, llamadas y papel	Duplicidad y pérdida de solicitudes	RF-03, RF-05 y RN-01
Inexistencia de un estado único de la solicitud	Docentes y responsables trabajan con información distinta	RF-05, RF-06 y RF-07
Aprobación sin control de rol ni de horario	Ocupación indebida y decisiones no auditables	RF-07, RN-04 y RN-05
Catálogo de ambientes desactualizado	Reservas sobre espacios inactivos o con datos erróneos	RF-02 y RN-02
Ausencia de indicadores de ocupación	Imposibilidad de planificar mantenimiento y apertura	RF-08

La literatura clásica de ingeniería de software respalda esta lectura del problema. McConnell (2004) señala que corregir un defecto en las etapas tempranas es significativamente más económico que hacerlo en producción, porque el costo crece con cada artefacto que depende del error. Myers, Sandler y Badgett (2011) añaden que el objetivo de la verificación temprana no es demostrar que el sistema funciona, sino descubrir las fallas antes de que el usuario final las sufra. En ReservaUPLA, esa premisa explica por qué las reglas de negocio se

especificaron antes de construir la interfaz y por qué cada iteración cerró con evidencia ejecutable.

1.2. Actores del sistema y matriz de responsabilidades

El caso práctico define tres actores mínimos. El análisis los conservó sin agregar roles artificiales, de modo que la matriz de permisos del software coincida exactamente con la estructura orgánica descrita.

Tabla 2. Matriz de actores, responsabilidades y permisos efectivos

Actor	Responsabilidad institucional	Permisos en el sistema	Restricciones verificadas
Solicitante	Registrar solicitudes de reserva para fines académicos y hacer seguimiento de su estado.	Crear solicitudes de reserva, consultar el listado de sus solicitudes personales («Mis Reservas»), verificar el detalle completo y cancelar sus propias solicitudes pendientes o aprobadas.	No aprueba, no rechaza, no administra usuarios ni ambientes, y no tiene visibilidad sobre las solicitudes privadas registradas por otros docentes (aislamiento de datos por usuario).
Administrador	Evaluar solicitudes, administrar el catálogo de ambientes y las cuentas de usuario, y supervisar indicadores.	Aprobar o rechazar reservas; gestionar usuarios y ambientes; consultar el dashboard completo.	No puede modificar una reserva rechazada o cancelada (RN-06).
Responsable de ambiente	Supervisar el uso programado de los ambientes bajo su custodia y planificar la operación logística.	Consultar el dashboard y el catálogo de ambientes con su ocupación.	No crea reservas ni accede a la gestión de usuarios.

La diferenciación de permisos no depende únicamente de ocultar botones. En el servicio `ReservaService.CambiarEstadoAsync` se recupera el usuario que ejecuta la acción y se comprueba que esté activo; si el nuevo estado es Aprobada o Rechazada y el actor no es Administrador, se lanza una excepción de acceso denegado. Del mismo modo, la cancelación solo se autoriza al administrador o al propio solicitante de la reserva. Esta decisión evita el escenario clásico en el que una petición HTTP manipulada omite las restricciones visuales.

La identificación de actores siguió el proceso de partes interesadas de la norma ISO/IEC/IEEE 12207:2017, que exige determinar quién solicita, quién opera y quién supervisa el sistema antes de definir requisitos. El caso práctico y las sesiones de clase coinciden en tres actores —Solicitante, Administrador y Responsable de ambiente—, y el análisis conservó exactamente esos tres, sin agregar roles que no participan en el proceso institucional.

1.3. Requerimientos funcionales atendidos

Los ocho requerimientos funcionales del caso práctico se implementaron y se verificaron contra la aplicación en ejecución. La tabla siguiente resume su materialización técnica y el punto de la interfaz donde puede comprobarse.

Tabla 3. Cobertura de los requerimientos funcionales RF-01 a RF-08

Código	Requerimiento	Implementación	Evidencia
RF-01	Registrar usuarios solicitantes	POST /api/usuarios con validación de correo institucional, rol y política de contraseña	Módulo Usuarios (solo Administrador)
RF-02	Registrar ambientes universitarios	POST /api/ambientes y edición mediante modal; control de nombre único y capacidad	Módulo Ambientes
RF-03	Registrar solicitudes de reserva	POST /api/reservas desde un modal sobre el listado	Figura 18
RF-04	Validar disponibilidad antes de registrar	Consulta de traslape aprobado y respuesta 409 con horario ocupado	Figura 22
RF-05	Listar reservas registradas	GET /api/reservas con filtros de estado, ambiente y fecha	Figura 33
RF-06	Consultar el detalle de una reserva	GET /api/reservas/{id} con datos del solicitante, ambiente y auditoría	Figura 36
RF-07	Cambiar el estado de una reserva	PATCH /api/reservas/{id}/estado con máquina de estados y control de rol	Figura 25
RF-08	Mostrar indicadores básicos	GET /api/dashboard con cinco indicadores consolidados	Figura 30

1.4. Alcance y límites del prototipo

El alcance comprometido comprende la autenticación con credenciales institucionales, la gestión de usuarios y ambientes, el ciclo completo de la reserva (registro, consulta, aprobación, rechazo y cancelación), la prevención de solapamientos, el tablero de indicadores y el registro de auditoría de las decisiones. También incluye recursos de accesibilidad —enlace para saltar al contenido, foco visible, trampa de foco en diálogos, mensajes anunciados con `aria-live` y modo de alto contraste— porque la aplicación será operada por personal con distintos niveles de familiaridad tecnológica.

Quedan explícitamente fuera del alcance: la integración con el sistema de horarios académicos, las notificaciones por correo electrónico o mensajería, la aplicación móvil nativa, la firma digital de autorizaciones y la facturación de espacios. Estos límites no se presentan como carencias, sino como decisiones de priorización coherentes con el objetivo de construir un prototipo funcional y verificable en nueve semanas.

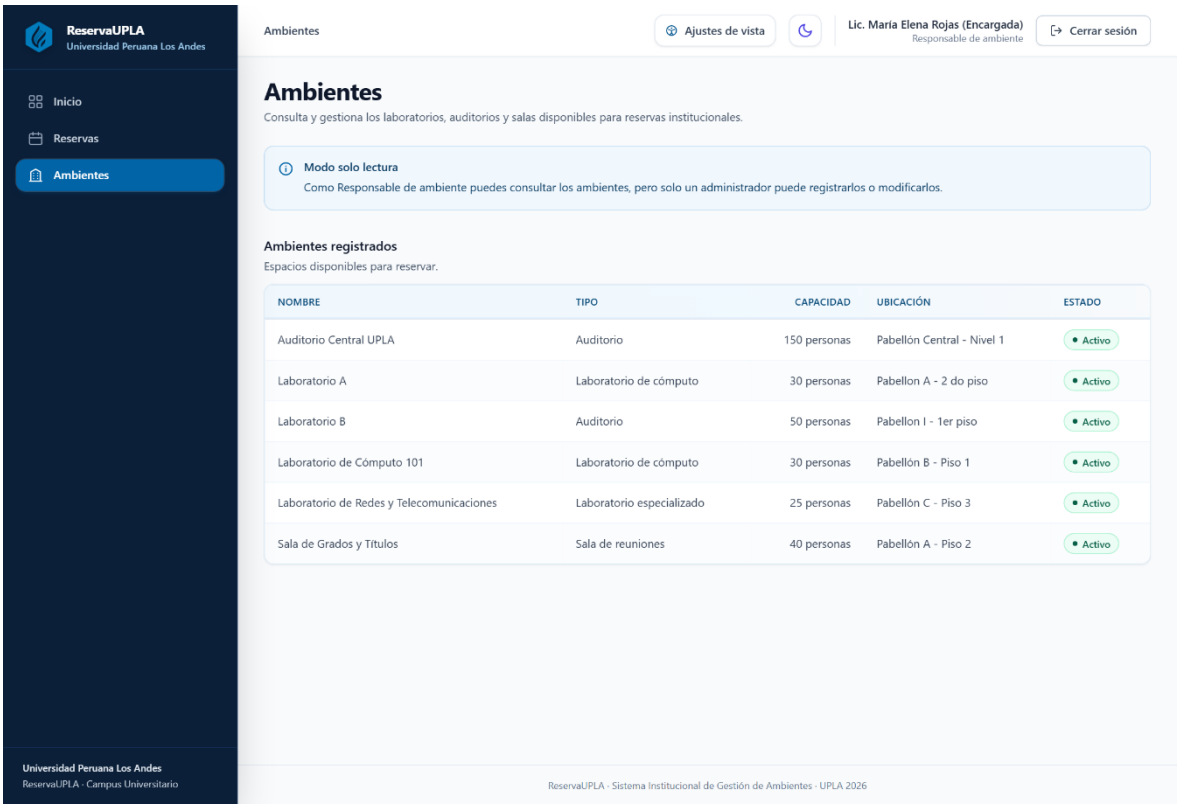


Figura 1. Vista del catálogo de ambientes desde el rol Responsable de ambiente

La Figura 1 evidencia la aplicación efectiva del principio de menor privilegio. El usuario autenticado como Responsable de ambiente accede al catálogo y al dashboard, pero el menú lateral no expone los módulos Usuarios ni la acción de crear reservas, pues su rol no participa en esas operaciones. La misma restricción se replica en el servidor: aunque se intentara invocar el endpoint de creación, la validación de identidad y rol impediría completar la operación.

2. SEMANA 02: CICLO DE VIDA DEL SOFTWARE

2.1. Modelo seleccionado

Se adoptó un modelo de ciclo de vida iterativo e incremental estructurado en nueve fases o iteraciones técnicas, diseñadas en estricta correspondencia con la progresión temática del caso práctico (Semanas 01 a 09 de la asignatura *Construcción de Software*).

La elección de este modelo responde a la naturaleza acumulativa de la ingeniería de software: en lugar de abordar el sistema mediante un enfoque tradicional en cascada (que posterga la integración y la prueba funcional hasta el final, elevando el riesgo de defectos estructurales), se definió que cada contenido temático del curso constituyera un incremento de software tangible y verificable. De este modo, el avance no se midió por horas o días cronológicos transcurridos, sino por el grado de completitud y madurez alcanzado en cada iteración técnica.

El modelo se adaptó a la dinámica de desarrollo individual (equipo unipersonal).

Cada ciclo cerró bajo un Definition of Done (Criterio de Aceptación) sustentado en cuatro condiciones binarias:

1. El código de la solución compila sin errores ni advertencias bloqueantes.
2. Las reglas de negocio de la fase se encuentran protegidas en el backend (servidor).
3. La interfaz de usuario proporciona retroalimentación clara y preventiva.
4. El comportamiento queda respaldado con evidencia ejecutable comprobable (capturas del sistema o consultas a la base de datos).

Este enfoque materializa los procesos del ciclo de vida de la norma **ISO/IEC/IEEE 12207:2017** (análisis, diseño, construcción y verificación) en cada iteración. Como señala McConnell (2004), la calidad no es una actividad que se inspecciona al término del proyecto, sino un atributo que se construye desde la primera línea de código.

2.2. Justificación técnica

La adopción de este enfoque iterativo por hitos temáticos se sustenta en cuatro argumentos técnicos comprobables en la arquitectura y en el repositorio del proyecto:

- **Incrementos progresivos y verificables:** En lugar de construir pantallas sin soporte o bases de datos sin conexión, cada fase añadió una capa funcional real: en la fase temática 04 se consolidó la solución en cuatro capas y la migración de base de datos; en la fase 05 los formularios procesaron entradas reales; en la fase 06 el núcleo validó y rechazó transacciones no permitidas; y en la fase 09 se cerró con el tablero de mando y auditoría.
- **Mitigación temprana de riesgos estructurales:** Las decisiones más críticas para la persistencia e integridad —tales como la restricción referencial (ON DELETE RESTRICT), el algoritmo de traslape horario con intervalos semiabiertos y la autorización en la capa de servicios— se resolvieron y probaron antes de diseñar las pantallas finales. Resolver un conflicto de concurrencia en la fase de arquitectura cuesta significativamente menos que corregirlo cuando el módulo de salida ya está construido.
- **Flexibilidad y retroalimentación de diseño (DCU):** La construcción por incrementos permitió evaluar la experiencia de usuario de manera temprana. Un ejemplo tangible fue la decisión de suprimir la pantalla independiente para registrar reservas: al notar que rompía la continuidad de trabajo del docente, se reemplazó por un diálogo modal

superpuesto sobre la grilla, alineándose con la regla de continuidad de contexto de Shneiderman.

- **Trazabilidad directa entre el sílabo académico y el código:** Cada requerimiento (RF-01 a RF-08) y regla de negocio (RN-01 a RN-06) está directamente mapeado a la fase de construcción que le dio origen. Esto permite que el presente informe técnico sustente cada afirmación con evidencia de código en ejecución y no con descripciones hipotéticas.

Tabla 4. Correspondencia entre fases temáticas, objetivos e incrementos de software entregados

Semana	Objetivo de la iteración	Incremento verificable
01	Comprender el problema, los actores y el alcance	Documento de análisis y matriz de actores
02	Seleccionar y justificar el ciclo de vida	Justificación técnica y plan de iteraciones
03	Definir estándares de código, nombres, carpetas y commits	Manual breve de estándares aplicado al repositorio
04	Construir la estructura backend, frontend y base de datos	Solución en cuatro capas, migraciones y modelo E-R
05	Implementar formularios de usuarios, ambientes y reservas	Formularios funcionales conectados a la API
06	Validar datos en frontend y backend	Reglas RN-01 a RN-06 con mensajes accionables
07	Reutilizar componentes de formulario y mensajes	Catálogo de nueve componentes integrados
08	Refactorizar formularios, servicios, controladores y validaciones	Código reorganizado y sin duplicación crítica
09	Construir listado, detalle y dashboard	Módulo de consulta y tablero con cinco indicadores

2.3. Gestión de riesgos de la iteración

Tabla 5. Riesgos identificados y estrategia de mitigación

Riesgo	Impacto potencial	Mitigación aplicada
Solapamiento de reservas aprobadas	Duplicidad de ocupación y conflictos entre docentes	Consulta de traslape con rangos semiabiertos y revalidación al aprobar (RN-01)
Eliminación accidental de ambientes con historial	Pérdida de trazabilidad y de registros de auditoría	Claves foráneas con `ON DELETE RESTRICT` y borrado lógico cuando existen reservas
Credenciales o cadena de conexión en el repositorio	Exposición de secretos institucionales	`dotnet user-secrets` para la cadena y la clave JWT; `.gitignore` para artefactos
Origen no autorizado consumiendo la API	Uso indebido de endpoints internos	CORS restringido a `http://localhost:5173` y autenticación JWT
Datos iniciales inconsistentes	Indicadores y listados sin sustento	Seeder idempotente y verificación con consultas directas a PostgreSQL

El Definition of Done de cada iteración exigió cuatro condiciones: el proyecto compila sin advertencias bloqueantes, las reglas de negocio involucradas se validan en el servidor, la interfaz presenta mensajes comprensibles y existe evidencia ejecutable del comportamiento. Esta última condición es la que permite que el presente informe muestre capturas del sistema y salidas reales de la base de datos en lugar de descripciones genéricas.

3. SEMANA 03: ESTÁNDARES DE CONSTRUCCIÓN DE SOFTWARE

3.1. Estándares de nomenclatura

Los estándares se definieron antes de escribir el código funcional y se aplicaron de manera uniforme. La tabla siguiente resume las convenciones adoptadas y un ejemplo real de cada una.

Tabla 6. Convenciones de nomenclatura por artefacto

Artefacto	Convención	Ejemplo en el proyecto
Clases y entidades C#	PascalCase, sustantivo singular	`ReservaService`, `Ambiente`, `BusinessRuleException`
Interfaces	Prefijo I + PascalCase	`IReservaRepository`, `IReservaService`
Métodos asíncronos	Verbo + complemento + sufijo Async	`CrearAsync`, `ObtenerTraslapeAprobadoAsync`
Campos privados	Guion bajo + camelCase	`_reservaRepository`, `_context`
Componentes React	PascalCase y extensión .jsx	`ModalNuevaReserva.jsx`, `CardIndicador.jsx`
Hooks y utilidades	camelCase y extensión .js	`useDialogAccessibility.js`, `validation.js`
Tablas y columnas PostgreSQL	PascalCase entre comillas	"Reservas", "HoraInicio", "UsuarioId"
Rutas de la API	Sustantivo en plural, minúsculas	`/api/reservas`, `/api/ambientes`, `/api/usuarios`

La decisión de conservar PascalCase en la base de datos merece una aclaración: PostgreSQL pliega los identificadores no citados a minúsculas, por lo que el mapeo de Entity Framework explicita los nombres entre comillas. Aunque existen convenciones que prefieren `snake_case`, se priorizó la correspondencia directa entre la entidad C# y la tabla, lo que reduce el riesgo de errores de mapeo y facilita la lectura de las consultas de verificación.

Los estándares de nomenclatura y organización responden al Principio de Responsabilidad Única formulado por Martin (2018): una clase debe tener una única razón para cambiar. En el proyecto, esa regla separa físicamente la entidad (datos), el servicio (lógica de negocio en ReservaService) y el repositorio (acceso a la base de datos). Fowler (2018) complementa el criterio al señalar que la documentación clara y el código legible facilitan la evolución, porque reducen el tiempo que una persona necesita para comprender qué hace cada pieza.

3.2. Estándares de arquitectura y organización de carpetas

El backend se organizó en cuatro proyectos que respetan la regla de dependencia de la arquitectura limpia: las capas externas conocen a las internas, nunca al revés. El dominio no referencia Entity Framework ni ASP.NET; la aplicación define contratos; la infraestructura implementa persistencia; y la API compone el sistema y expone los endpoints.

Tabla 7. Responsabilidades y dependencias de las capas

Proyecto	Responsabilidad	Depende de
ReservaUPLA.Domain	Entidades, constantes de estado y roles; no contiene dependencias externas	Ninguna
ReservaUPLA.Application	DTOs, interfaces de repositorio y servicio, excepciones de negocio, servicios de caso de uso	Domain
ReservaUPLA.Infrastructure	DbContext, configuraciones de mapeo, repositorios concretos, generación de JWT, seeder	Application y Domain
ReservaUPLA.API	Controladores delgados, middleware de excepciones, configuración de seguridad y Swagger	Application e Infrastructure

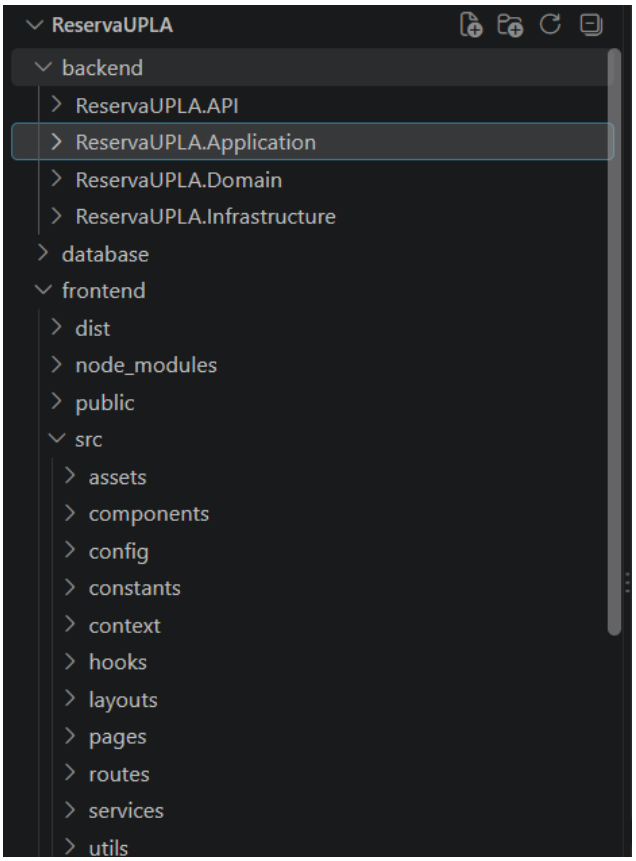


Figura 2. Estructura del repositorio y organización de carpetas del proyecto (Backend en Arquitectura Limpia y Frontend React)

3.3. Estándares de versionado y commits

Se adoptó **Conventional Commits** por su capacidad de generar historiales legibles y de vincular cada cambio con un tipo de modificación. Los mensajes se redactan en modo imperativo y en español, con un alcance opcional entre paréntesis. La rama principal concentra las versiones estables y cada funcionalidad se desarrolla en una rama corta que se integra tras verificar la compilación y el comportamiento.

Tabla 8. Tipos de commit y uso previsto

Tipo	Uso	Ejemplo
feat	Nueva funcionalidad visible para el usuario	`feat(reservas): agregar modal de nueva solicitud`
fix	Corrección de un defecto	`fix(auth): mantener sesión al recargar la página`
refactor	Reorganización sin cambio de comportamiento	`refactor(http): centralizar el manejo de errores`
style	Cambios de formato o presentación	`style(dashboard): ajustar contraste de indicadores`
docs	Documentación del proyecto	`docs(informe): documentar reglas de negocio`
test	Pruebas automatizadas	`test(reservas): cubrir el traslape de horarios`

3.4. Estándares de interfaz y accesibilidad

La interfaz se construyó con utilidades de Tailwind y un conjunto reducido de componentes con variantes, lo que evita hojas de estilo dispersas. Se fijaron criterios de espaciado en múltiplos de 4 px, una paleta institucional con azul marino como color primario y escalas tipográficas consistentes. Los estados de foco son visibles, los controles tienen etiqueta asociada y los diálogos atrapan el foco mientras permanecen abiertos.

Estos criterios se alinean con los principios de diálogo de la norma ISO 9241-110 —adecuación a la tarea, autodescripción, conformidad con las expectativas, tolerancia al error y control por parte del usuario— y con las heurísticas de Nielsen, en particular la visibilidad del estado del sistema, la prevención de errores y la ayuda para reconocer y recuperarse de los errores.

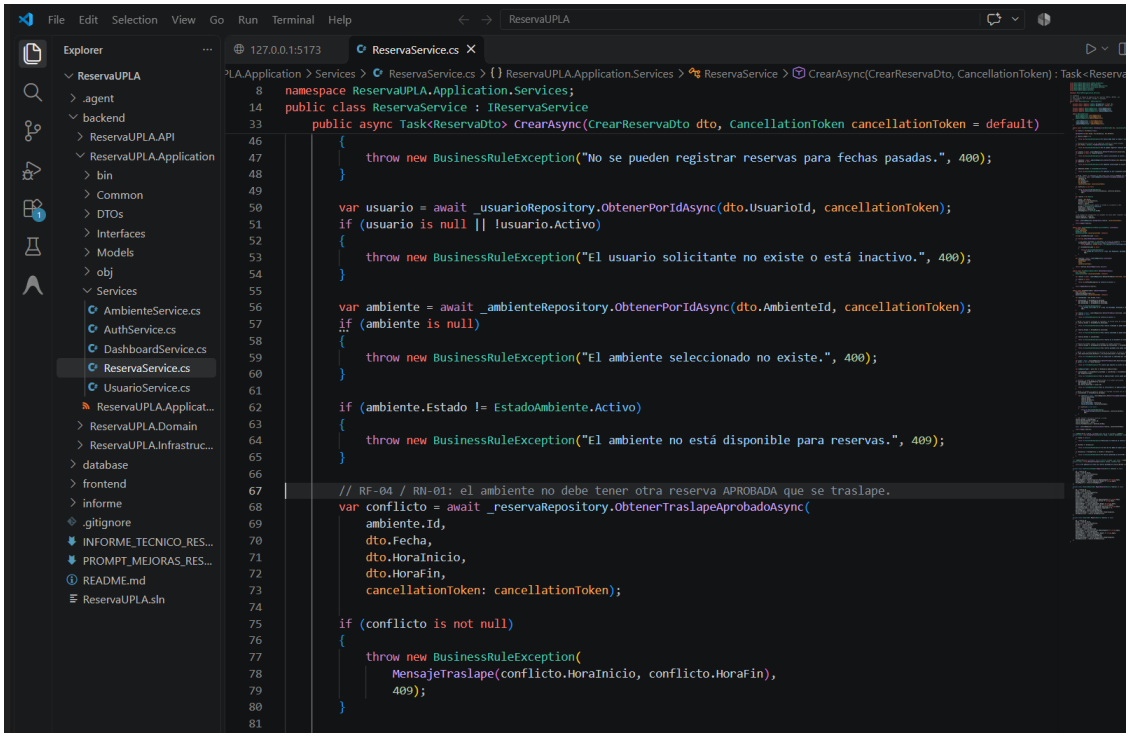


Figura 3. Estructura del repositorio y lógica de reservas en el editor

La Figura 3 muestra dos elementos que sustentan los estándares descritos: el árbol del repositorio con la separación física de las capas y el archivo `ReservaService.cs`, donde las reglas de negocio se concentran en la capa de aplicación. Los controladores no contienen lógica de decisión; reciben la petición, delegan en el servicio y devuelven una respuesta uniforme. Esta delgadez facilita las pruebas y evita que una modificación de política institucional obligue a editar varios controladores.

4. SEMANA 04: ARQUITECTURA Y MODELO DE DATOS

4.1. Arquitectura de la solución

La arquitectura implementada es una arquitectura limpia en cuatro capas con inyección de dependencias. El flujo de una petición típica es el siguiente: el controlador recibe el DTO, invoca la interfaz del servicio registrada en el contenedor, el servicio aplica las reglas y utiliza los repositorios para persistir o consultar, y el middleware traduce cualquier excepción de negocio a una respuesta JSON con el código HTTP correspondiente. Ninguna capa externa accede directamente al `DbContext` salvo la infraestructura.

Esta separación tiene una consecuencia práctica verificable: las reglas RN-01 a RN-06 residen en `ReservaService` y no en los controladores ni en la interfaz. Si mañana se agregara un cliente móvil o una integración con otro sistema, las reglas se reutilizarían sin reescribirlas. La interfaz, por su parte, replica las validaciones para ofrecer retroalimentación inmediata, pero nunca se considera la única barrera.

La separación en capas materializa el mismo principio en escala arquitectónica: el dominio no conoce la base de datos, la aplicación no conoce el controlador y la infraestructura no conoce la interfaz. Martin (2018) denomina a esta dirección de dependencias regla de dependencia y advierte que invertirla convierte al dominio en rehén de los detalles. Myers, Sandler y Badgett (2011) agregan que una arquitectura con límites claros permite probar cada capa por separado y detectar los defectos antes de que se propaguen al conjunto.

4.1.1. Stack tecnológico seleccionado y justificación técnica

Para materializar la arquitectura limpia y garantizar el rendimiento, la mantenibilidad y la seguridad del sistema ReservaUPLA, se seleccionó un ecosistema tecnológico moderno, maduro y de código abierto. La tabla siguiente consolida cada capa de la solución, las herramientas implementadas y el fundamento técnico de su elección.

Tabla 9b. Matriz del Stack Tecnológico del Sistema ReservaUPLA

Capa / Dominio	Tecnología / Herramienta	Versión	Rol en ReservaUPLA	Justificación Técnica de Elección
Backend Framework	**ASP.NET Core Web API**	**.NET 8.0**	Núcleo de la API RESTful y orquestación de servicios	Alto rendimiento de procesamiento asíncrono (`async/await`), soporte nativo para Inyección de Dependencias (DI) y tipado estricto en C#.
Acceso a Datos (ORM)	**Entity Framework Core**	**8.0**	Mapeo objeto-relacional y ejecución de migraciones	Facilita el modelado en C# mediante Code-First, control de transacciones y optimización de consultas de solo lectura con `AsNoTracking()`.
Base de Datos	**PostgreSQL**	**16.x**	Motor de base de datos relacional institucional	Soporte nativo para tipos UUID, claves foráneas con `ON DELETE RESTRICT`, restricciones `CHECK` horarias y robustez ACID probada.
Frontend Library	**React**	**18 / 19**	Construcción de la interfaz de usuario interactiva	Modelo declarativo basado en componentes reutilizables, gestión eficiente del DOM virtual y actualización reactiva de estados locales.
Herramienta de Build	**Vite**	**5.x / 8.x**	Empaquetador y servidor de desarrollo ultrarrápido	Arranque instantáneo gracias al uso de módulos ES nativos (ESM) y Hot Module Replacement (HMR) optimizado para desarrollo.
Diseño y Estilos	**Tailwind CSS**	**3.4.x**	Sistema de diseño y maquetación responsiva	Enfoque *utility-first* que evita el crecimiento desordenado de archivos CSS, estandariza escalas de 4 px y facilita el soporte de modo oscuro.
Enrutamiento	**React Router DOM**	**6.x**	Navegación del lado del cliente (SPA)	Permite la navegación fluida entre vistas (Inicio, Ambientes, Reservas, Detalle) sin recargar la página, alimentando las migas de pan.
Seguridad y Criptografía	**BCrypt.Net-Next**	**4.0.3**	Hashing seguro de contraseñas de usuarios	Algoritmo con sal (*salt*) adaptativo resistente a ataques de fuerza bruta y tablas arcoíris (*rainbow tables*), cumpliendo estándares OWASP.
Autenticación	**JWT (JSON Web Tokens)**	**Bearer**	Control de sesiones sin	Emisión de tokens firmados mediante HMAC-SHA256

			estado (Stateless)	con claims de identidad y rol (RBAC) para consumo seguro desde el cliente HTTP.
Documentación de API	**Swagger / OpenAPI**	**6.x**	Contrato de interfaz y pruebas interactivas	Generación automática de la especificación técnica interactiva, exponiendo rutas, DTOs y respuestas HTTP (200, 201, 400, 409).

4.2. Modelo entidad-relación

El modelo conserva las tres entidades mínimas del caso práctico y amplía la entidad Reserva con los campos necesarios para la auditoría: `ResueltoPorId` y `FechaResolucion`. Asimismo, agrega `PasswordHash` a Usuario para almacenar credenciales de forma segura y `FechaRegistro` a las tres entidades para conservar la marca temporal de creación.



Figura 4. Modelo entidad-relación implementado en PostgreSQL

La relación entre Usuario y Reserva se presenta dos veces: una por la solicitud (`UsuarioId`) y otra por la resolución (`ResueltoPorId`). Ambas son relaciones uno a muchos y utilizan borrado restringido. Ambiente se relaciona con Reserva mediante `AmbienteId`, también con borrado restringido. El diseño impide que la eliminación de un usuario o de un ambiente destruya el historial de reservas, condición indispensable para la trazabilidad institucional.

4.3. Diccionario de datos

Las tablas siguientes documentan las columnas efectivamente creadas por las migraciones de Entity Framework Core y verificadas con `psql`. Se indica el tipo PostgreSQL, la obligatoriedad, la restricción asociada y la finalidad del campo.

Tabla 9. Diccionario de datos de la tabla Usuarios

Columna	Tipo	Nulo	Restricción	Descripción
Id	uuid	No	PK	Identificador único del usuario
NombreCompleto	varchar(150)	No	—	Nombre y apellidos del usuario
Correo	varchar(150)	No	UK	Correo institucional, usado como credencial
Rol	varchar(50)	No	—	Solicitante, Administrador o Responsable de ambiente
PasswordHash	varchar(255)	No	—	Hash BCrypt de la contraseña
Activo	boolean	No	—	Habilita o suspende el acceso
FechaRegistro	timestamptz	No	—	Momento de alta del usuario

Tabla 10. Diccionario de datos de la tabla Ambientes

Columna	Tipo	Nulo	Restricción	Descripción
Id	uuid	No	PK	Identificador único del ambiente
Nombre	varchar(100)	No	UK	Nombre institucional del espacio
Tipo	varchar(50)	No	—	Laboratorio, auditorio, sala u otro
Capacidad	integer	No	—	Aforo en personas
Ubicacion	varchar(100)	No	—	Pabellón y nivel
Estado	varchar(30)	No	—	Activo, Inactivo o Mantenimiento
Activo	boolean	No	—	Marca de vigencia del registro

Tabla 11. Diccionario de datos de la tabla Reservas

Columna	Tipo	Nulo	Restricción	Descripción
Id	uuid	No	PK	Identificador único de la reserva
Fecha	date	No	—	Fecha de uso del ambiente
HoraInicio	time	No	CHECK	Hora de inicio, entre 07:00 y 22:00
HoraFin	time	No	CHECK	Hora de fin, mayor que la de inicio
Motivo	text	No	—	Justificación académica o institucional
Estado	varchar(30)	No	—	Pendiente, Aprobada, Rechazada o Cancelada
UsuarioId	uuid	No	FK	Solicitante de la reserva
AmbienteId	uuid	No	FK	Ambiente reservado
ResueltoPorId	uuid	Sí	FK	Administrador que aprobó, rechazó o canceló
FechaRegistro	timestamptz	No	—	Momento de creación de la solicitud
FechaResolucion	timestamptz	Sí	—	Momento de la decisión

4.4. Integridad, índices y reglas en la base de datos

La integridad no se delegó exclusivamente a la aplicación. El esquema incorpora restricciones `CHECK` para el rango horario y para la relación entre hora de inicio y hora de fin, además de índices que sostienen las consultas más frecuentes. Esta decisión protege los datos incluso frente a scripts de mantenimiento o cargas masivas que no pasen por la API.

Tabla 12. Restricciones e índices del esquema ReservaUPLADB

Objeto	Definición	Propósito
CK_Reservas_HoraFin_Mayor	CHECK ("HoraFin" > "HoraInicio")	Garantiza RN-03 en el almacenamiento
CK_Reservas_Rango_Permitido	CHECK ("HoraInicio" >= '07:00' AND "HoraFin" <= '22:00')	Garantiza RN-04 en el almacenamiento
IX_Reservas_Ambiente_Fecha_Estado	Índice compuesto (AmbienteId, Fecha, Estado)	Acelera la consulta de traslape de RN-01
IX_Reservas_Fecha	Índice simple (Fecha)	Optimiza listados y filtros por fecha
IX_Reservas_UsuarioId	Índice simple (UsuarioId)	Optimiza la consulta de reservas por solicitante
FK_Reservas_Usuarios_UsuarioId	FOREIGN KEY con ON DELETE RESTRICT	Protege el historial ante bajas de usuarios
FK_Reservas_Ambientes_AmbienteId	FOREIGN KEY con ON DELETE RESTRICT	Protege el historial ante bajas de ambientes
FK_Reservas_Usuarios_ResueltoPorId	FOREIGN KEY con ON DELETE RESTRICT	Conserva la identidad del auditor

El índice compuesto `IX_Reservas_Ambiente_Fecha_Estado` es especialmente relevante porque la consulta de solapamiento filtra simultáneamente por ambiente, fecha y estado. Sin ese índice, cada intento de reserva recorrería la tabla completa; con él, el motor localiza únicamente las reservas aprobadas del ambiente en la fecha consultada. La medición de

tiempos no forma parte de este informe, pero la estructura del plan de consulta justifica la decisión de diseño.

4.5. Evidencia de la base de datos implementada

La verificación se realizó con `psql` sobre la base `ReservaUPLADB`, utilizando el usuario de aplicación `reservaupla_app`. La salida confirma la existencia de las tres tablas de negocio, la volumetría real y la distribución de reservas por estado. No se empleó pgAdmin porque no se encuentra instalado en el equipo; la consola de PostgreSQL constituye una evidencia equivalente y más reproducible.

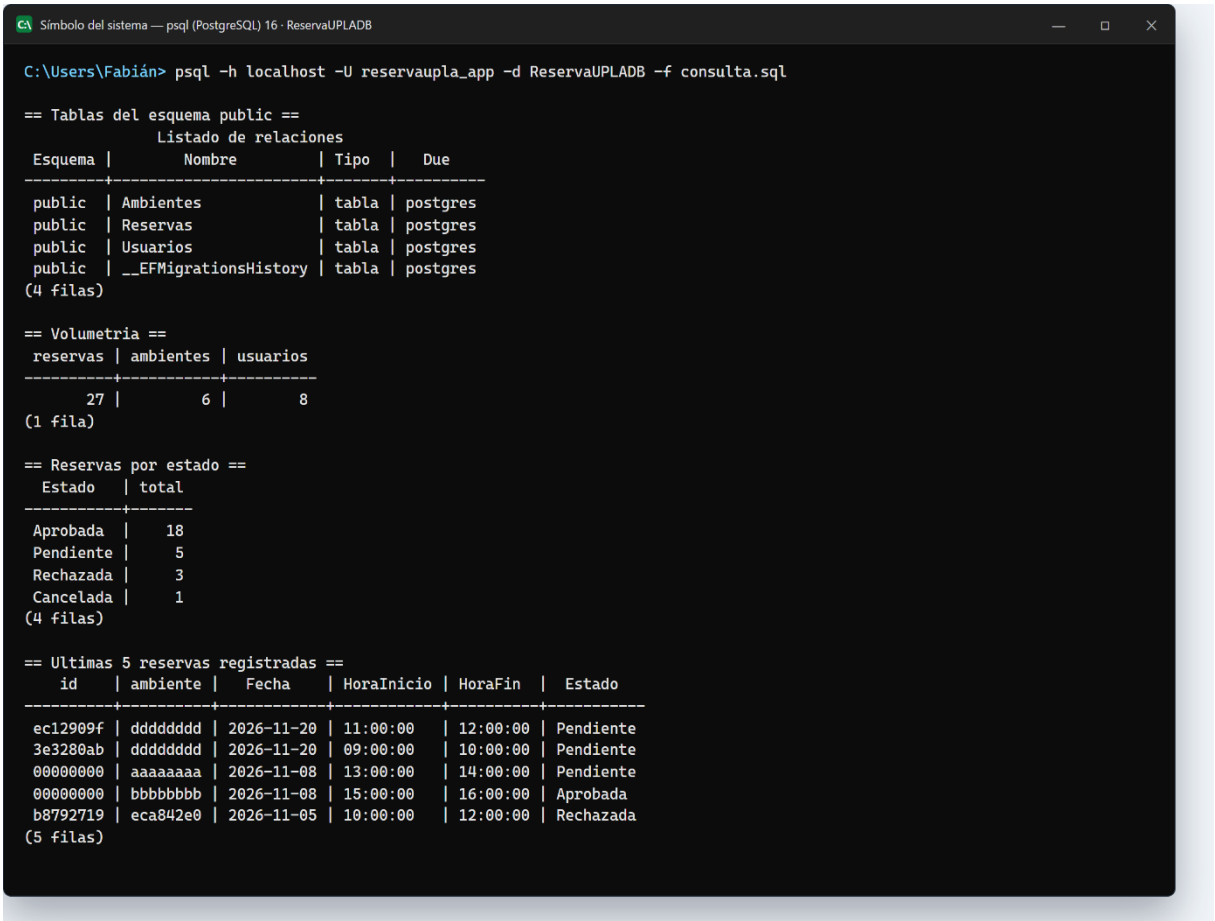


Figura 5. Verificación de tablas, volumetría y distribución de estados en PostgreSQL

4.6. Seguridad de la persistencia y de la autenticación

Las contraseñas se almacenan como hash BCrypt (BCrypt.Net-Next 4.0.3); nunca se guarda la contraseña en claro ni se devuelve en las respuestas. La autenticación emite un JWT firmado con HMAC-SHA256 que incluye los claims de identificador, correo, nombre y rol, con una vigencia predeterminada de ocho horas. El frontend conserva el token en `sessionStorage` y lo adjunta en la cabecera `Authorization` mediante un cliente HTTP centralizado.

La cadena de conexión y la clave de firma se administran con `dotnet user-secrets`, de modo que no forman parte del repositorio. El middleware de excepciones registra el detalle técnico con `ILogger` y devuelve al cliente un mensaje genérico, práctica alineada con OWASP. El inicio de sesión utiliza el mismo mensaje para credenciales inexistentes y para contraseñas incorrectas, con el fin de evitar la enumeración de usuarios.

5. DISEÑO PREVIO DE LA INTERFAZ Y WIREFRAMES

5.1. Enfoque de diseño centrado en el usuario

Antes de implementar las pantallas se elaboraron wireframes de media fidelidad con el objetivo de validar la estructura, la jerarquía visual y el flujo de las tareas principales. El proceso siguió el enfoque de diseño centrado en el usuario de la norma ISO 9241-210: comprender el contexto de uso, especificar los requisitos, producir soluciones de diseño y evaluarlas. Los wireframes no son decorativos: cada uno responde a una decisión de interacción que luego se verificó en el sistema construido.

Se adoptaron tres principios rectores. La continuidad de contexto evita que el usuario pierda el lugar donde estaba trabajando; por ello la creación de reservas ocurre en un diálogo sobre el listado y no en una página independiente. La prevención antes que la corrección anticipa los errores mediante controles restringidos y mensajes junto al campo. La consistencia asegura que los mismos componentes se comporten igual en todos los módulos, de modo que aprender una pantalla equivalga a aprender el sistema.

5.2. Wireframe de la pantalla de acceso

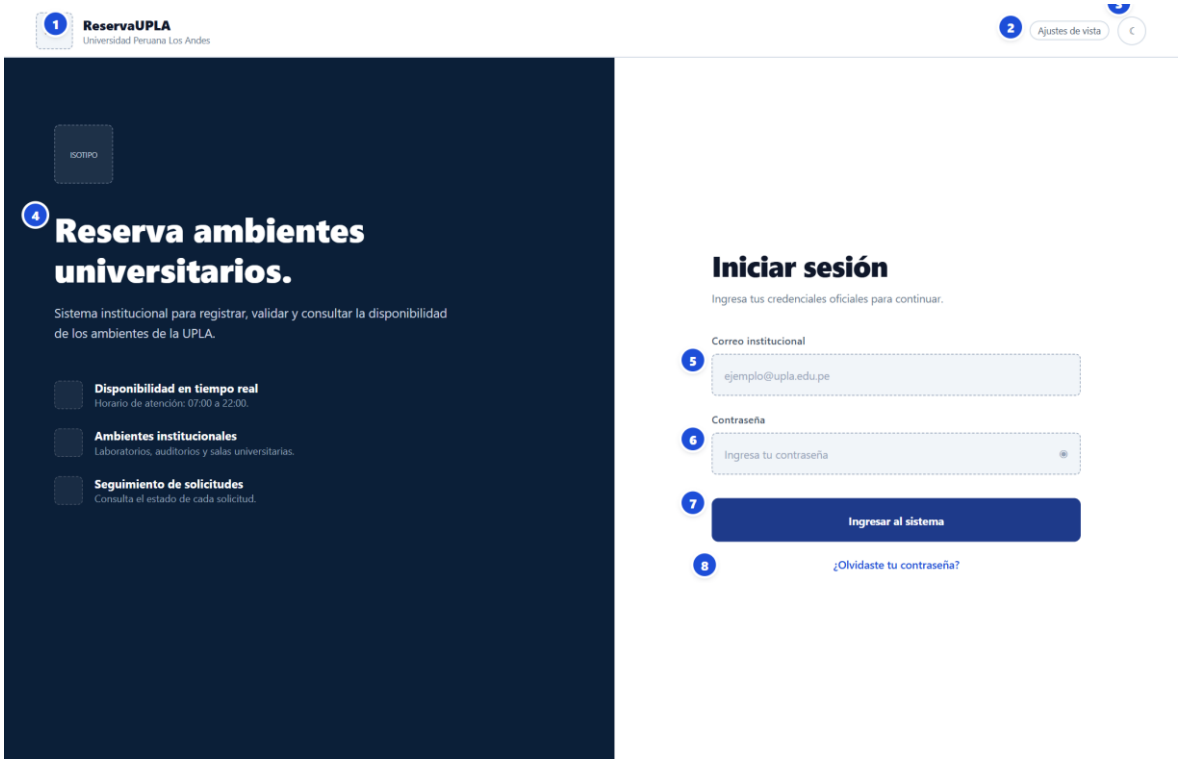


Figura 6. Wireframe anotado de la pantalla de inicio de sesión

La Figura 6 presenta el wireframe anotado del acceso institucional. Cada llamada numerada corresponde a una fila de la Tabla 13, que documenta el comportamiento esperado del control y la heurística, regla o criterio que sostiene la decisión de diseño.

Tabla 13. Especificación del wireframe de inicio de sesión

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Encabezado de marca	Logotipo, nombre del sistema y texto alternativo; no es interactivo.	Nielsen 2: correspondencia con el mundo real. WCAG 1.1.1: alternativa textual.
②	Ajustes de vista	Despliega texto ampliado, alto contraste y restablecer; guarda la preferencia en el dispositivo.	Nielsen 7: flexibilidad y eficiencia. WCAG 1.4.4: redimensionar el texto.
③	Conmutador de tema	Alterna la clase oscura en el documento y conserva la elección; contraste AA en ambos temas.	Nielsen 1: visibilidad del estado. WCAG 1.4.3: contraste mínimo.
④	Panel institucional	Tres mensajes fijos de contexto; no compite con el formulario.	Nielsen 8: diseño minimalista. Shneiderman 8: reducir la carga de memoria.
⑤	Correo institucional	Tipo correo, obligatorio, formato institucional; error en línea con aria-describedby.	Nielsen 5: prevención de errores. WCAG 3.3.1 y 3.3.2: error identificado y descrito.
⑥	Contraseña	Campo enmascarado con botón de visibilidad (aria-pressed); no interfiere con el gestor de contraseñas.	Nielsen 5: el usuario decide cómo verificar lo que escribe. WCAG 1.4.3.
⑦	Ingresar al sistema	Acción primaria única; estado de carga y bloqueo durante el envío; foco visible.	Shneiderman 1: consistencia. Nielsen 1: visibilidad del estado del sistema.
⑧	Recuperación de acceso	Enlace secundario, sin peso de acción primaria; informa el canal institucional.	Nielsen 6: reconocer antes que recordar. Jerarquía visual.

La decisión de mayor impacto es la ausencia de un selector de rol: el rol proviene del registro del usuario y no de una declaración del cliente, de modo que la interfaz no ofrece una elección

que el servidor ignoraría. Los controles de accesibilidad y de tema se ubican en la barra superior, fuera del recorrido principal, para no competir con la tarea de ingreso.

5.3. Wireframe del Inicio

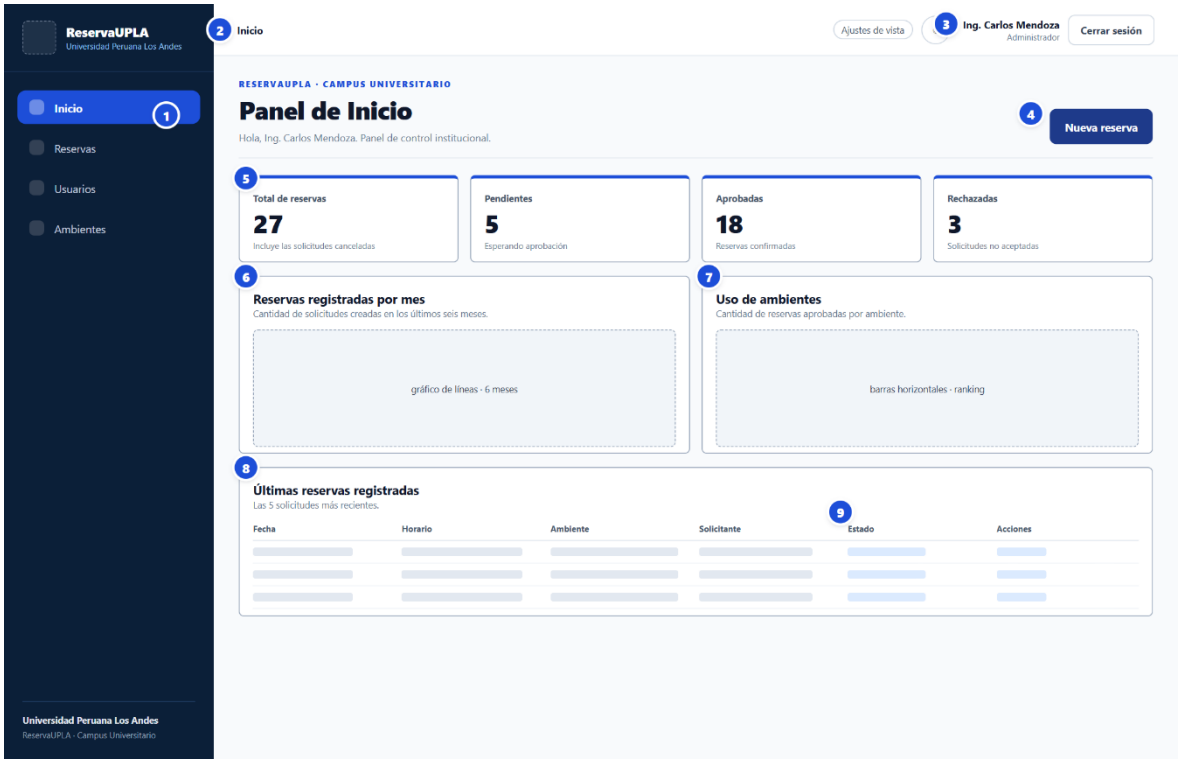


Figura 7. Wireframe anotado del tablero institucional

Tabla 14. Especificación del wireframe del tablero institucional.

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Navegación lateral	Enlaces filtrados por rol; estado activo con aria-current; enlace para saltar al contenido.	Nielsen 4: consistencia y estándares. Shneiderman 3: control del usuario.
②	Migas de pan	Ruta jerárquica generada desde la URL; no duplica el menú.	Nielsen 1: orientación permanente. WCAG 2.4.8: ubicación.
③	Identidad y sesión	Nombre, rol y cierre de sesión; el token se conserva en sessionStorage.	Nielsen 1: el sistema informa quién opera y con qué permisos.
④	Nueva reserva	Visible solo para Solicitante y Administrador; abre el diálogo sin abandonar el tablero.	Shneiderman 2: atajos para usuarios frecuentes. Nielsen 7.
⑤	Tarjetas de indicadores	Cuatro métricas desde la API; esqueleto de carga; descripción textual del valor.	Nielsen 1 y WCAG 1.1.1: el dato no depende del color.
⑥	Serie mensual	Gráfico SVG con role de imagen y etiqueta accesible; seis meses sobre la fecha de registro.	WCAG 1.1.1: alternativa textual para gráficos.
⑦	Uso de ambientes	Barras ordenadas e insignia con el ambiente más utilizado sobre reservas aprobadas.	Nielsen 2: el ranking se lee en el lenguaje del dominio.
⑧	Últimas reservas	Cinco filas con fecha, horario, ambiente, solicitante, motivo y estado.	Nielsen 6: reconocimiento del contexto reciente.
⑨	Badges de estado	Pendiente, Aprobada, Rechazada y Cancelada con color y texto, nunca solo color.	WCAG 1.4.1: el color no es el único medio de información.

El tablero se organizó en cuatro niveles de lectura: identificación y acción principal, indicadores cuantitativos, visualizaciones de tendencia y detalle reciente. La jerarquía permite responder primero «cuánto hay» y después «dónde y cuándo», sin obligar al usuario a interpretar gráficos antes de conocer las cifras.

El rótulo del módulo se unificó como «Inicio» y el encabezado como «Panel de Inicio». La decisión responde a la heurística 2 de Nielsen (2020), correspondencia entre el sistema y el mundo real: el personal administrativo de la UPLA reconoce «Inicio» y no el anglicismo «Dashboard», que obliga a traducir el vocabulario técnico antes de actuar. La miga de pan conserva la misma raíz y los indicadores mantienen sus nombres institucionales —Total de reservas, Pendientes, Aprobadas y Rechazadas—, de modo que toda la pantalla habla el idioma del usuario y no el del desarrollador.

5.4. Wireframe del catálogo y gestión de ambientes

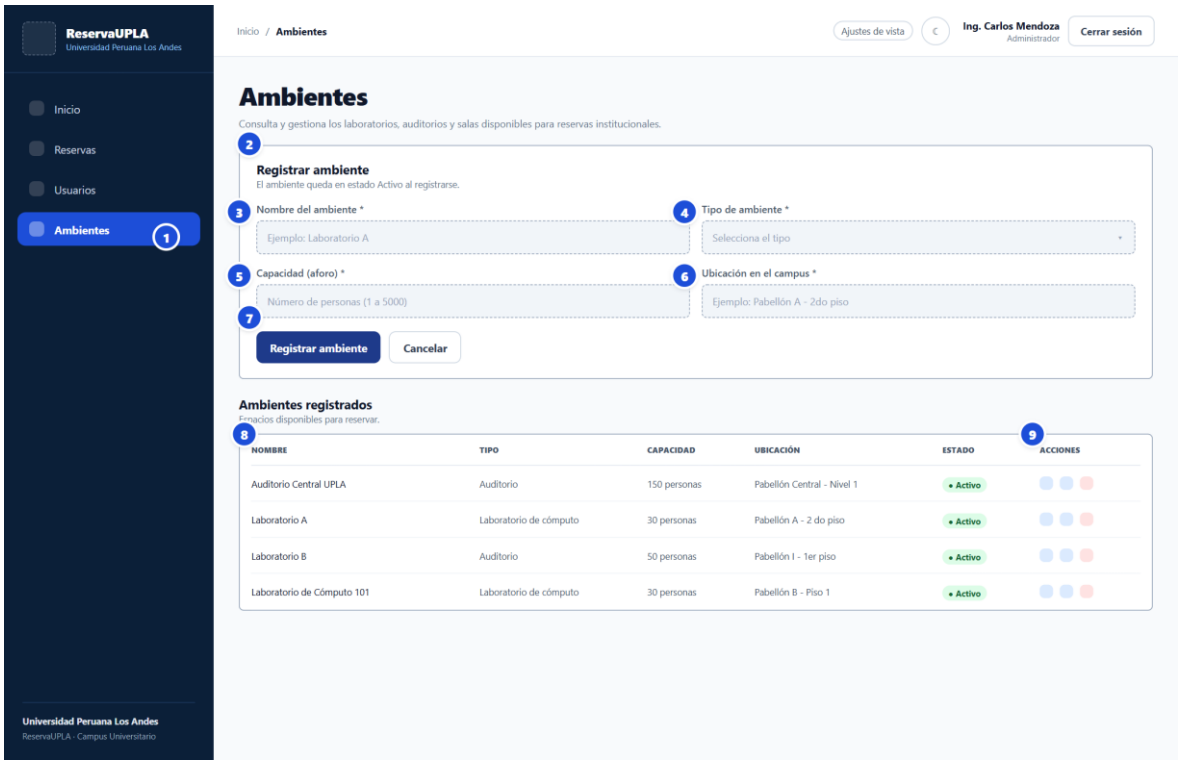


Figura 8. Wireframe anotado del catálogo y la gestión de ambientes

La Figura 8 presenta el wireframe anotado del módulo de ambientes. Las nueve llamadas se especifican en la Tabla 15 y cubren el alta, la tabla con estados y las acciones protegidas por la integridad referencial.

Tabla 15. Especificación del wireframe del catálogo de ambientes

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Navegación lateral	Enlaces filtrados por rol; el Responsable de ambiente no ve Usuarios ni acciones de edición.	Nielsen 4: consistencia. Menor privilegio aplicado a la interfaz.
②	Formulario de alta	Nombre, tipo, capacidad, ubicación y estado inicial Activo decidido por el sistema.	Nielsen 2: lenguaje del dominio. Shneiderman 3: control del usuario.
③	Nombre del ambiente	Obligatorio, entre 3 y 100 caracteres, único sin distinguir mayúsculas ni tildes.	Nielsen 5: prevención de errores. WCAG 3.3.1.
④	Tipo de ambiente	Selector con laboratorio de cómputo, laboratorio especializado, auditorio y sala.	Nielsen 6: opciones visibles, sin texto libre.
⑤	Capacidad	Entero entre 1 y 5000; el servidor revalida y responde 400 fuera del rango.	Nielsen 5 y validación en dos niveles (semana 06).
⑥	Ubicación	Entre 3 y 100 caracteres; se muestra en el listado y en el detalle.	Nielsen 2: información completa para decidir.
⑦	Acciones del formulario	Registrar como acción primaria y Cancelar como secundaria; bloqueo durante el envío.	Shneiderman 1: consistencia. Nielsen 1: estado del sistema.
⑧	Tabla de ambientes	Badges Activo, Inactivo y Mantenimiento; nunca solo color.	WCAG 1.4.1: el color no es el único medio.
⑨	Acciones por fila	Editar, cambiar estado y eliminar; la baja se bloquea si existen reservas (ON DELETE RESTRICT).	Nielsen 5: prevención de pérdida de historial.

El módulo separa dos responsabilidades: administrar el catálogo y proteger el historial. La eliminación no destruye un ambiente con reservas porque la clave foránea restringida lo impide en la base de datos, y la interfaz lo advierte antes de intentarlo. El Responsable de ambiente accede al mismo catálogo en modo de consulta, lo que confirma que el control de acceso se aplica por rol y no por pantalla.

5.5. Wireframe de la gestión y el registro de usuarios

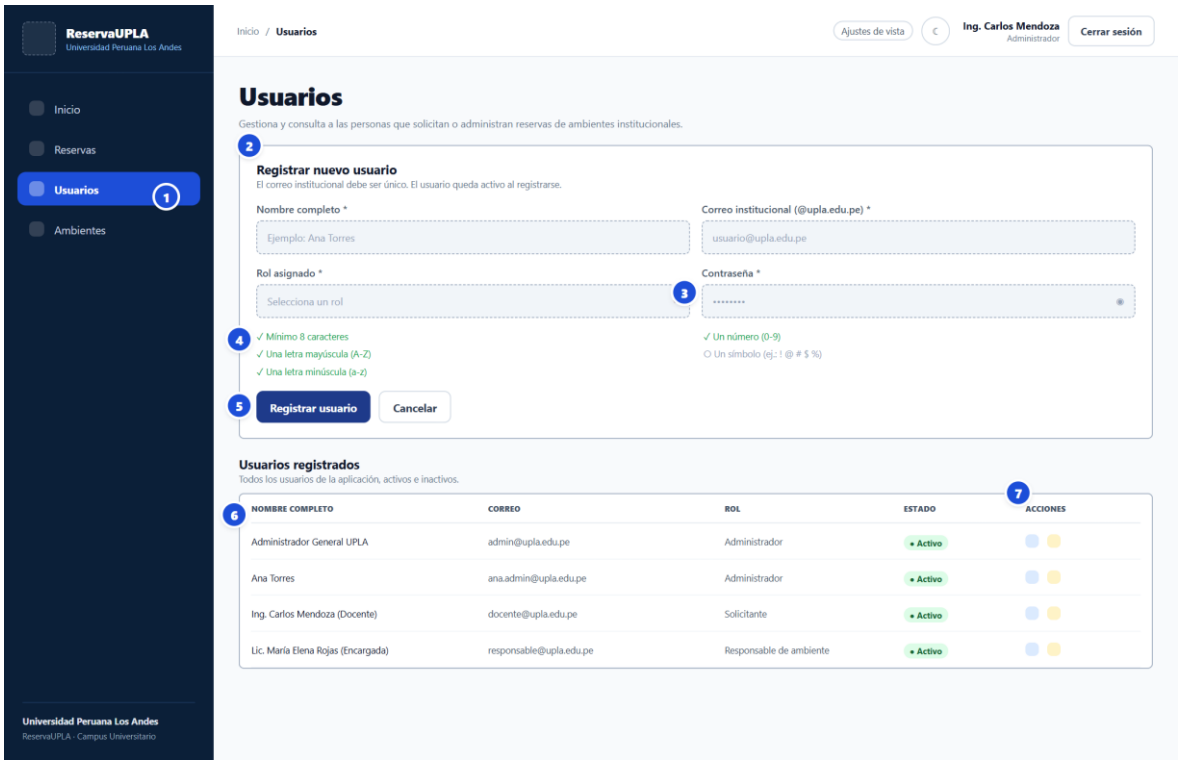


Figura 9. Wireframe anotado de la gestión y el registro de usuarios

La Figura 9 presenta el wireframe anotado del módulo de usuarios. Las siete llamadas se especifican en la Tabla 16 y cubren el alta, la política de contraseña y las acciones de mantenimiento de cuentas.

Tabla 16. Especificación del wireframe de gestión de usuarios

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Menú lateral	Muestra Inicio, Reservas, Usuarios y Ambientes; el módulo Usuarios solo aparece para el Administrador.	Control de acceso basado en roles (RBAC). Nielsen 4: consistencia.
②	Formulario de alta	Nombre completo, correo institucional y rol obligatorios; el rol se elige de una lista cerrada.	Nielsen 2: lenguaje del dominio. Shneiderman 3: control del usuario.
③	Correo institucional	Valida el dominio @upla.edu.pe y la unicidad; el servidor responde 409 si el correo ya existe.	Nielsen 5: prevención de errores. WCAG 3.3.1.
④	Contraseña enmascarada	Campo tipo password con botón SVG de visibilidad (aria-pressed); no interfiere con el gestor de contraseñas.	Nielsen 5 y WCAG 1.4.3: el usuario decide cómo verificar lo que escribe.
⑤	Requisitos en tiempo real	Cinco marcas (ocho caracteres, mayúscula, minúscula, número y símbolo) que cambian mientras se escribe.	McConnell (2004): validación temprana. Nielsen 5: el error se impide antes del envío.
⑥	Registrar usuario	Acción primaria con bloqueo anti-doble envío y estado de carga; la misma política se revalida en el servidor.	Nielsen 1: visibilidad del estado. Shneiderman 1: consistencia.
⑦	Tabla y acciones	Nombre, correo, rol y estado Activo/Inactivo; editar datos y suspender o reactivar el acceso.	Nielsen 6: reconocimiento. Trazabilidad del estado de la cuenta.

El módulo aplica el control de acceso basado en roles y concentra la política de contraseña en un solo componente reutilizable. La lista de requisitos se muestra antes del envío y la regla se revalida en el servidor con PasswordPolicy, de modo que la seguridad no depende del navegador. Las cuentas no se eliminan: se suspenden, para conservar la trazabilidad de las reservas que registraron.

5.6. Wireframe del registro de reserva

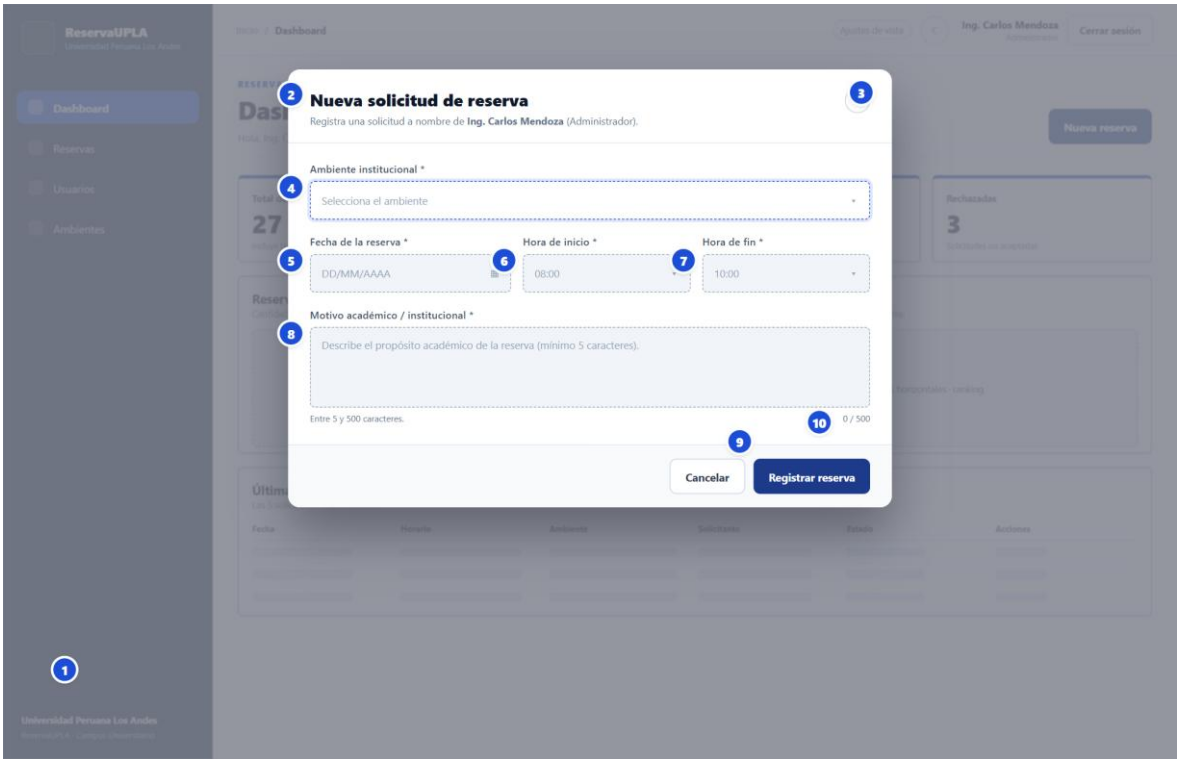


Figura 10. Wireframe anotado del diálogo de registro de una nueva reserva

La Figura 10 presenta el wireframe anotado del diálogo de registro. Las diez llamadas se especifican en la Tabla 17, que relaciona cada control con su validación y con el criterio de usabilidad que la fundamenta.

Tabla 17. Especificación del wireframe del diálogo de reserva

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Superposición	Atenúa el listado y bloquea la interacción de fondo; conserva el contexto de la pantalla.	Shneiderman 6: continuidad de contexto. Nielsen 3: control y libertad.
②	Título y destinatario	Indica la acción y a nombre de quién se registra; el solicitante proviene de la sesión.	Nielsen 2: lenguaje del usuario. Impide la suplantación.
③	Cerrar	Botón con etiqueta accesible «Cerrar modal»; Escape cierra salvo durante el envío.	WCAG 2.1.2: sin trampas de teclado. Nielsen 3.
④	Ambiente	Selector accesible por teclado; seis ambientes activos; campo obligatorio.	Nielsen 6: opciones visibles. WCAG 2.1.1: operable con teclado.
⑤	Fecha	Mínimo igual al día actual; rechaza fechas pasadas con mensaje junto al campo.	Nielsen 5: prevención de errores. WCAG 3.3.1.
⑥	Hora de inicio	Formato de 24 horas acotado a la jornada de 07:00 a 22:00 (RN-04).	Nielsen 5 y RN-04: el control evita el valor inválido.
⑦	Hora de fin	Mínimo dinámico posterior al inicio; validación en línea (RN-03).	Nielsen 5: el error se impide antes de enviar.
⑧	Motivo y contador	Entre 5 y 500 caracteres; contador en vivo y mensaje anunciado con aria-live.	Nielsen 1 y WCAG 3.3.2: estado y ayuda simultáneos.
⑨	Acciones	Cancelar como acción secundaria y Registrar como primaria; bloqueo durante el envío.	Shneiderman 1: consistencia. Nielsen 5: una acción principal por pantalla.
⑩	Foco y teclado	Foco inicial en el primer campo; trampa de foco; retorno al elemento que abrió el diálogo.	WCAG 2.4.3: orden del foco. WCAG 2.1.2.

El diálogo reúne los cinco datos que exige RN-02 y conserva el listado al fondo, atenuado, como referencia de contexto. Fecha, hora de inicio y hora de fin comparten una fila porque forman una unidad conceptual; el motivo ocupa el ancho completo por ser texto libre. La operación se bloquea durante el envío para evitar registros duplicados.

5.7. Wireframe del listado de reservas con filtros

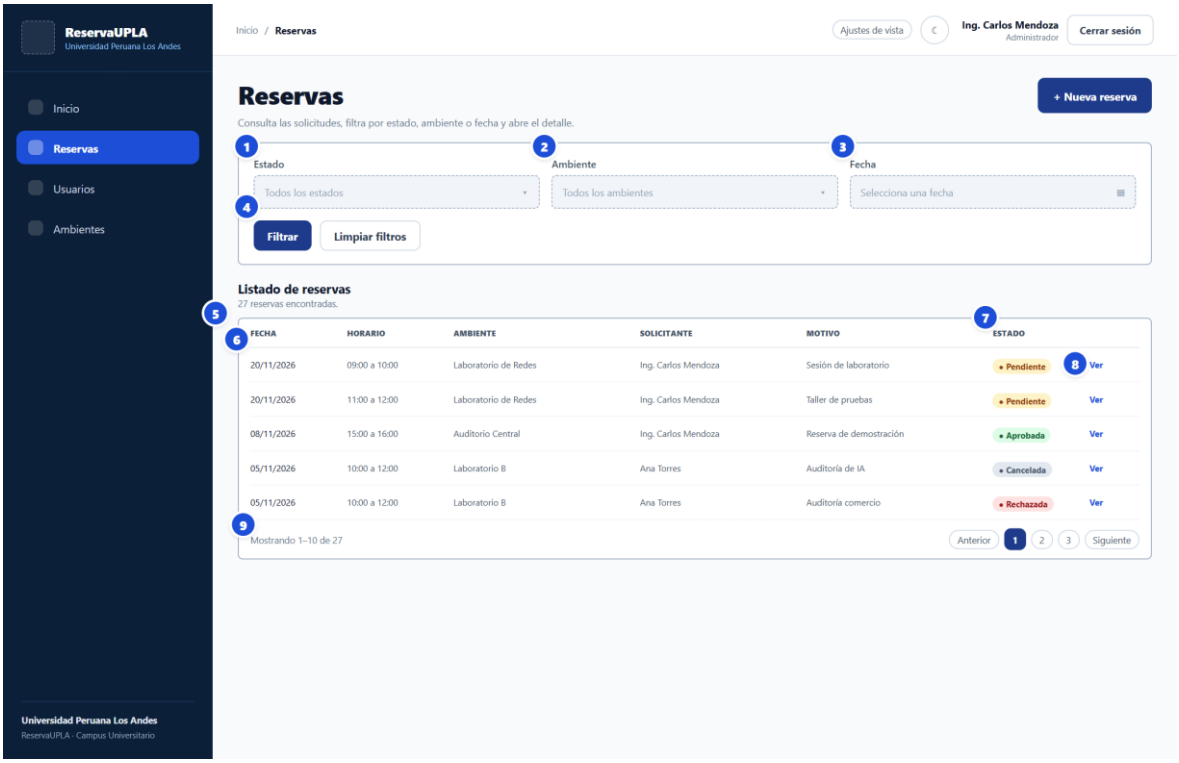


Figura 11. Wireframe anotado del listado de reservas con filtros combinables

La Figura 11 presenta el wireframe anotado del listado. Las nueve llamadas se detallan en la Tabla 18 y ordenan la lectura desde los criterios de búsqueda hasta la paginación.

Tabla 18. Especificación del wireframe del listado de reservas

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Filtro de estado	Pendiente, Aprobada, Rechazada o Cancelada; combinable con los demás filtros.	Nielsen 6: reconocer las opciones. Shneiderman 3.
②	Filtro de ambiente	Catálogo de seis ambientes; se envía como parámetro de consulta.	Nielsen 7: eficiencia para usuarios frecuentes.
③	Filtro de fecha	Selector con calendario; vacío significa todas las fechas.	Nielsen 5: el rango válido se ofrece en el control.
④	Filtrar y limpiar	Aplica los criterios o los restablece sin recargar la página.	Shneiderman 3: control y libertad del usuario.
⑤	Contador de resultados	«27 reservas encontradas»: confirma el efecto del filtro antes de leer la tabla.	Nielsen 1: visibilidad del estado del sistema.
⑥	Columnas del listado	Fecha, horario, ambiente, solicitante, motivo y estado; motivos truncados con acceso al detalle.	Nielsen 8: diseño minimalista, sin ruido visual.
⑦	Badge de estado	Traduce el valor de dominio a texto y color; nunca muestra códigos.	Nielsen 2: correspondencia con el mundo real.
⑧	Acción Ver	Abre el detalle conservando los filtros aplicados.	Shneiderman 6: continuidad de contexto.
⑨	Paginación	Diez registros por página, indicador de rango y navegación anterior/siguiente.	Nielsen 1 y 3: ubicación y control.

El listado resuelve el problema de localizar una solicitud entre decenas de registros sin obligar a recorrer la tabla completa. Los filtros se combinan y el contador confirma el resultado de la combinación antes de leerla; el detalle se abre conservando el contexto, de modo que volver al listado no obliga a repetir la búsqueda.

5.8. Wireframe de la vista de detalle y auditoría

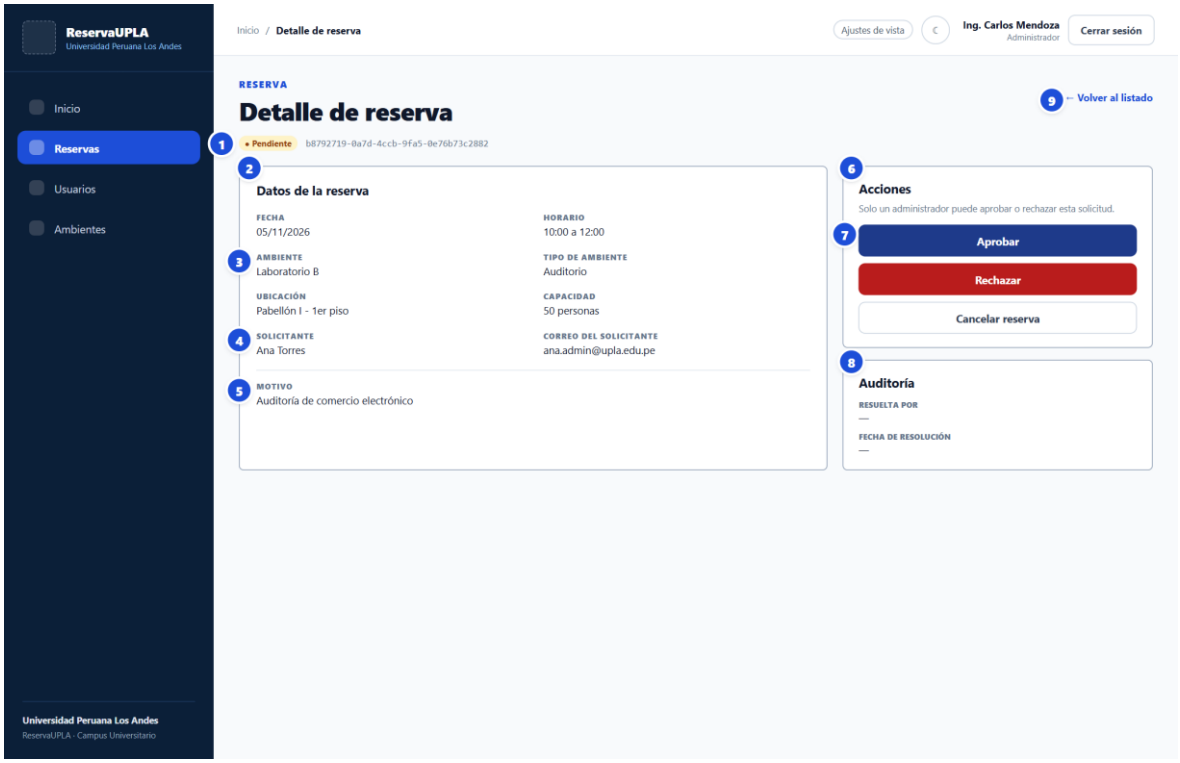


Figura 12. Wireframe anotado de la vista de detalle y del panel de auditoría *Fuente: elaboración propia.*

La Figura 12 presenta el wireframe anotado del detalle. Las nueve llamadas se especifican en la Tabla 19 y separan los datos de la solicitud de las acciones que la resuelven.

Tabla 19. Especificación del wireframe del detalle de reserva

N°	Componente UI	Comportamiento / validación técnica	Justificación DCU / usabilidad
①	Estado y folio	Badge del estado actual y identificador completo de la reserva.	Nielsen 1: el usuario sabe en qué estado está el registro.
②	Datos de la reserva	Fecha, horario, ambiente, capacidad y tipo en una grilla de dos columnas.	Nielsen 8: lectura agrupada por proximidad.
③	Ambiente	Nombre y ubicación física para verificar el espacio asignado.	Nielsen 2: información del dominio institucional.
④	Solicitante	Nombre y correo de quien registró la solicitud, tomados de la sesión.	Trazabilidad: impide la suplantación.
⑤	Motivo	Texto completo, sin truncar, en un bloque separado.	Nielsen 6: el contexto completo evita juicios erróneos.
⑥	Panel de acciones	Visible solo para Administrador; informa la restricción de rol.	Nielsen 4 y RN-05: coherencia entre permisos y acciones.
⑦	Aprobar, Rechazar y Cancelar	Cada acción abre una confirmación; el estado final se escribe en el servidor.	Nielsen 5: prevención de decisiones irreversibles.
⑧	Auditoría	Resuelta por y fecha de resolución; se completa al aprobar, rechazar o cancelar.	Trazabilidad institucional y confianza en el dato.
⑨	Volver al listado	Retorna conservando los filtros previos.	Shneiderman 6: continuidad de contexto.

El detalle es la pantalla que sostiene la trazabilidad. Los datos permanecen separados de las acciones y el panel de auditoría se completa en el servidor con el actor autenticado y la marca temporal, de modo que la afirmación «esta reserva fue resuelta por esta persona en este momento» se apoya en el registro y no en una suposición.

5.9. Trazabilidad entre wireframe e implementación

Tabla 20. Decisiones de diseño y su correspondencia con la implementación

Decisión de diseño	Fundamento	Implementación verificable
Acción primaria única por pantalla	Jerarquía visual y prevención de errores	Botón «Ingresar al sistema»; «Nueva reserva»; «Registrar reserva»
Formulario de acceso sin selector de rol	El rol proviene del registro, no del cliente	`AuthService` obtiene el usuario y firma el claim de rol
Creación de reserva en modal	Continuidad de contexto (Shneiderman)	`ModalNuevaReserva` sobre `ReservasPage`
Agrupación fecha–inicio–fin	Proximidad y lectura natural del rango	Tres columnas en una misma fila del formulario
Indicadores antes que gráficos	Lectura de lo general a lo particular	Cuatro tarjetas seguidas de dos gráficos
Navegación filtrada por rol	Menor privilegio y reducción de ruido	`MainLayout` filtra enlaces según `rolActual`

6. SEMANA 05: MÓDULOS DE ENTRADA DE DATOS

Los módulos de entrada constituyen la puerta de acceso de la información al sistema. Se construyeron tres formularios funcionales —usuarios, ambientes y reservas— además del formulario de autenticación. Todos comparten los mismos componentes de campo, la misma lógica de validación y el mismo tratamiento de errores, lo que produce una experiencia homogénea y reduce el costo de mantenimiento.

6.1. Formulario de acceso

El formulario de acceso solicita correo institucional y contraseña. Al enviar, el cliente invoca `POST /api/auth/login`; el servicio normaliza el correo, verifica el hash BCrypt y, si las credenciales son válidas, devuelve el token junto con los datos del usuario. La sesión se restaura al recargar la página consultando el perfil con el token almacenado, de modo que el usuario no pierde su trabajo por una actualización accidental del navegador.

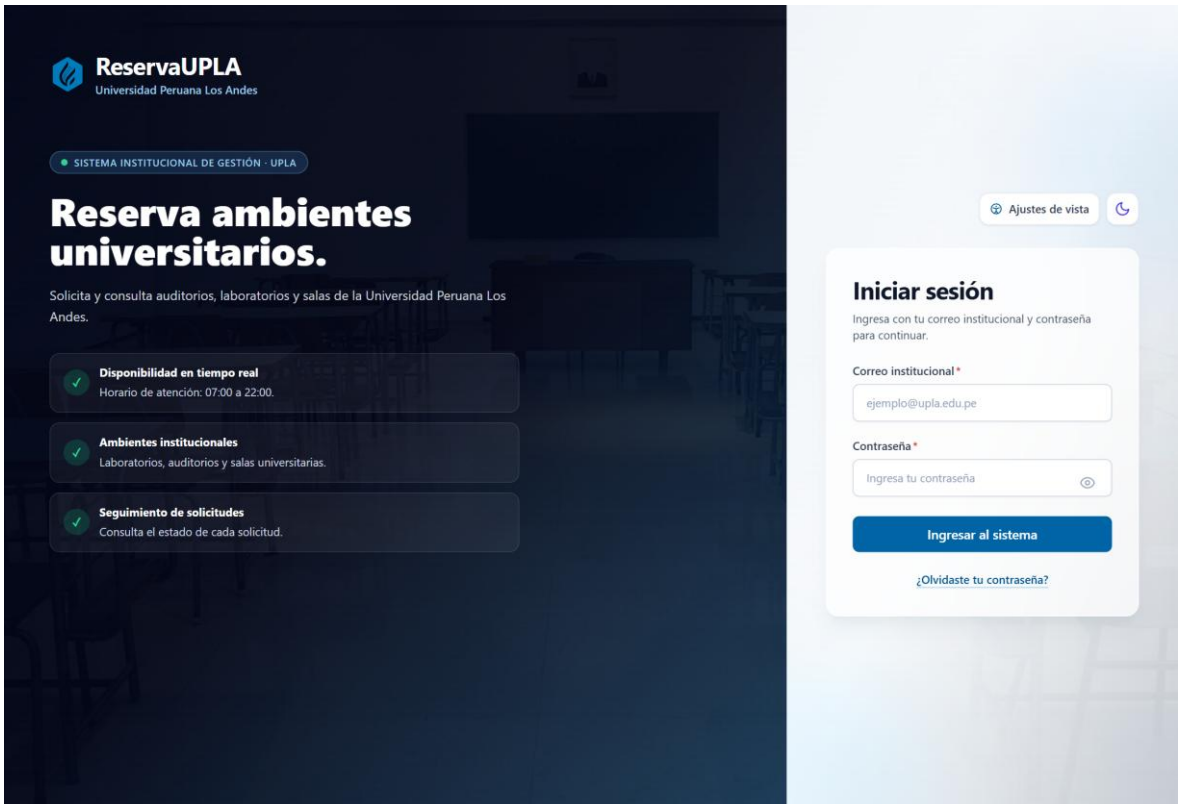


Figura 13. Pantalla de inicio de sesión implementada

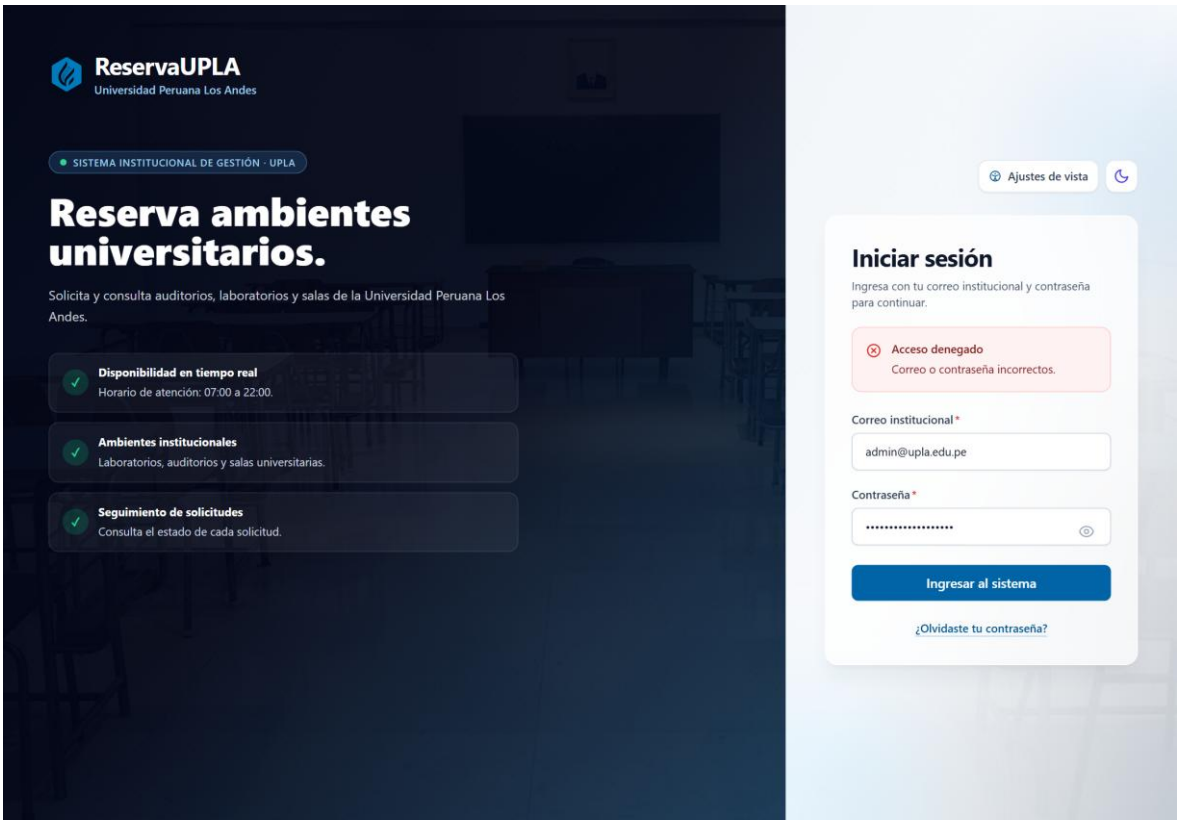


Figura 14. Validación de credenciales incorrectas en el acceso

El sistema rechaza el intento con un mensaje único para correo inexistente o contraseña errónea, de modo que no revela cuál de los dos datos falló. La heurística 9 de Nielsen se cumple porque el aviso explica el problema y permite reintentar sin abandonar la pantalla, mientras que la práctica de OWASP evita la enumeración de usuarios.

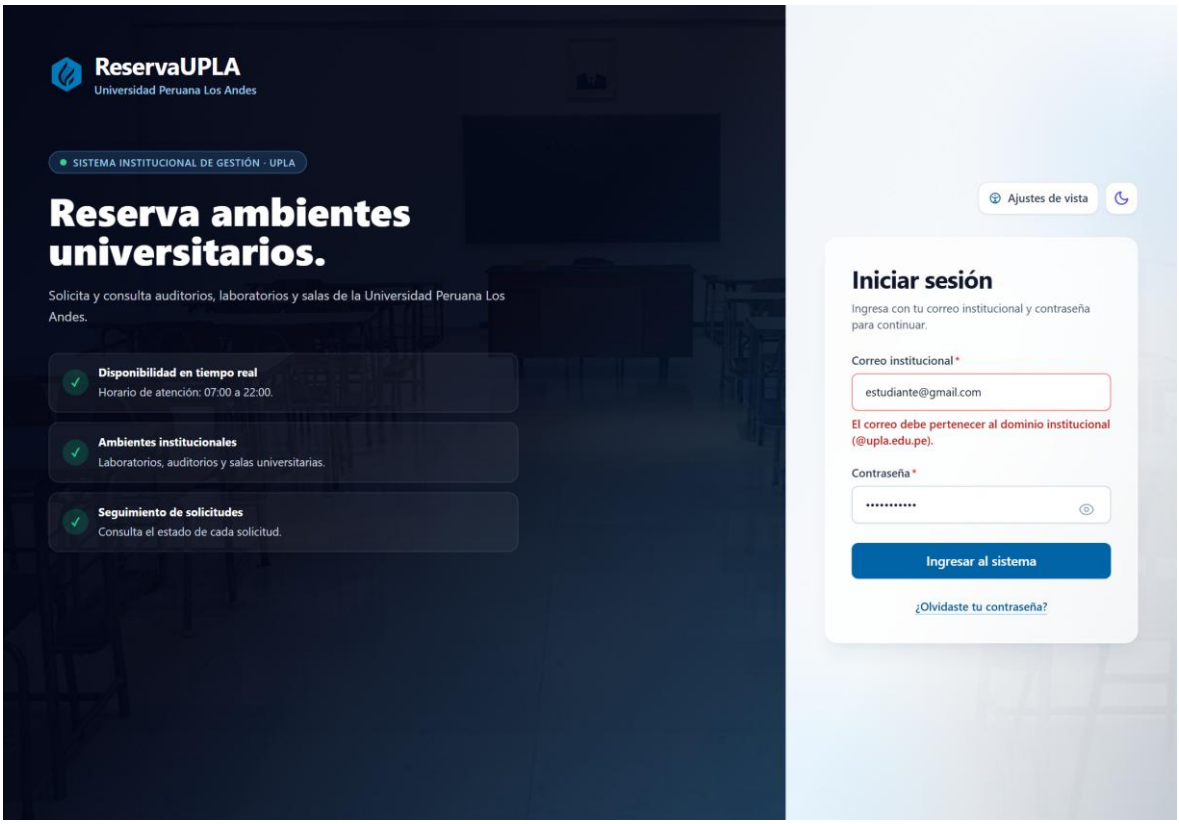


Figura 15. Restricción del correo al dominio institucional (@upla.edu.pe)

La validación ocurre antes de enviar la petición: el campo se marca en rojo y el mensaje indica el dominio esperado. Se evita así una llamada innecesaria al servidor y el usuario corrige el dato en el mismo lugar donde lo escribió.

6.2. Formulario de ambientes

El formulario de ambientes captura nombre, tipo, capacidad, ubicación y estado operativo. La capacidad se valida como entero entre 1 y 5000 personas; el nombre se controla para evitar duplicados y se compara sin distinguir mayúsculas ni tildes mediante una función de normalización. El estado inicial lo decide el sistema —todo ambiente nuevo nace Activo— y no se acepta un estado impuesto por el cliente, práctica que evita registros inconsistentes.

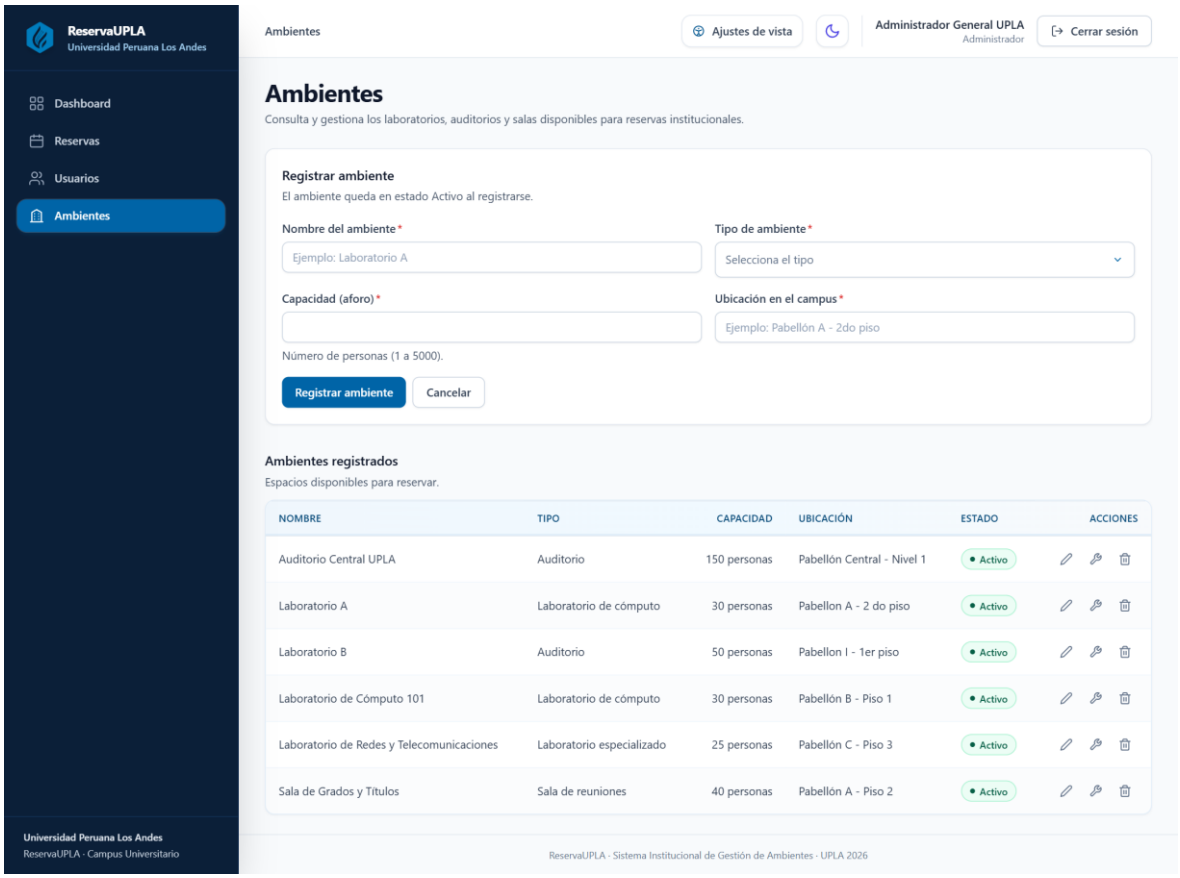


Figura 16. Formulario de registro y edición de ambientes universitarios

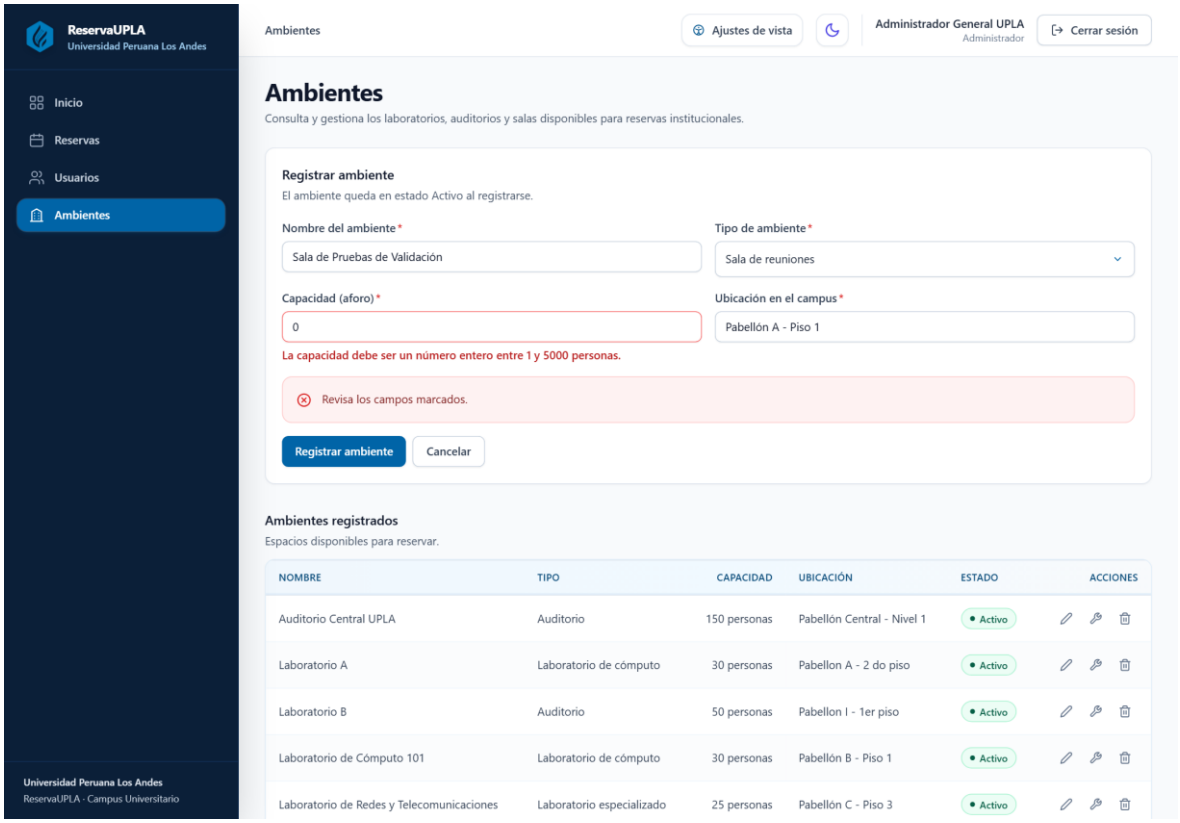


Figura 17. Validación del rango de capacidad del ambiente

La capacidad se declara entre 1 y 5000 personas en la interfaz y se revalida en el servicio, que responde 400 si el valor no cumple. El formulario conserva los datos para corregir y el registro no se crea, lo que demuestra la doble barrera de validación.

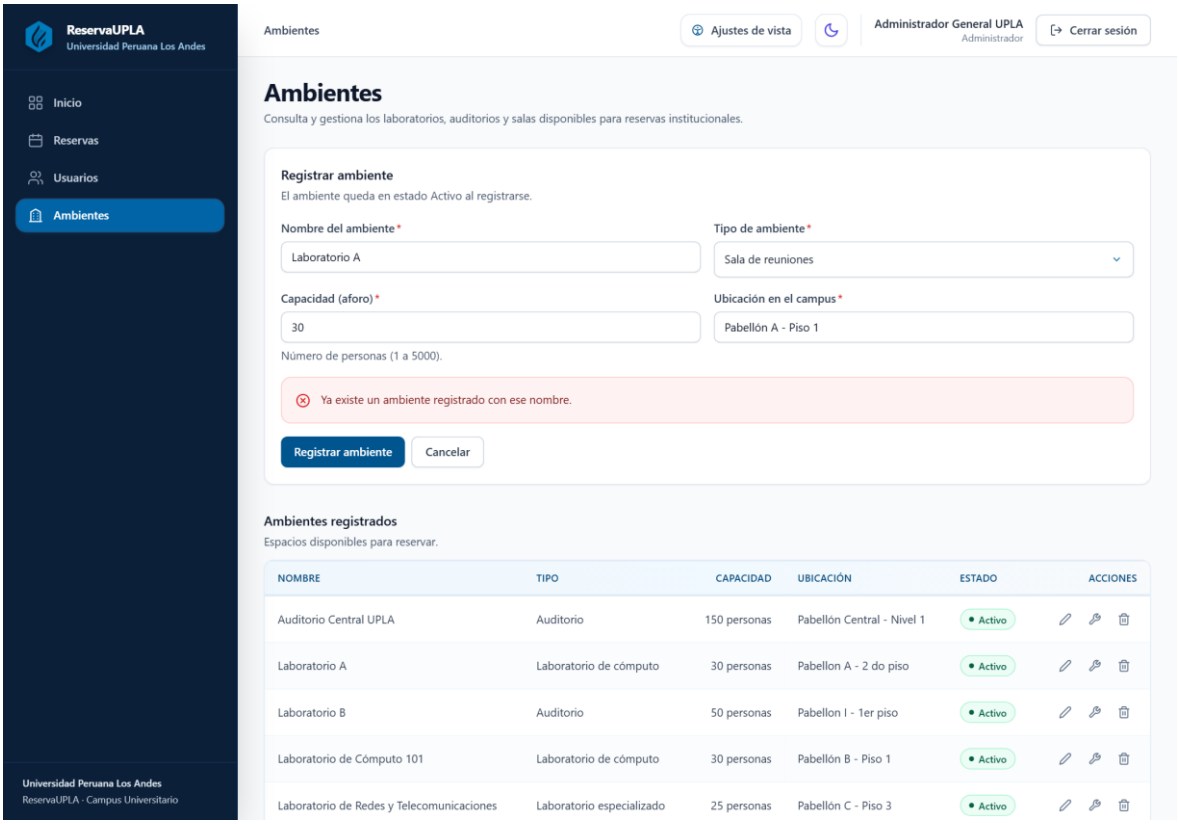


Figura 18. Validación de unicidad del nombre de ambiente

El servicio detecta el nombre repetido y devuelve un conflicto que la interfaz traduce en un mensaje comprensible. La tabla mantiene visible el ambiente original, de modo que el usuario comprende por qué el alta no procede y puede corregir el nombre.

6.3. Formulario de reservas

El formulario de reservas es el módulo de entrada de mayor responsabilidad porque alimenta el núcleo del negocio. Solicita ambiente, fecha, hora de inicio, hora de fin y motivo. La fecha mínima es el día actual; la hora de fin se habilita después de elegir la de inicio y su valor mínimo se desplaza dinámicamente; el motivo exige entre 5 y 500 caracteres y muestra un contador en tiempo real. El usuario solicitante no se elige en el formulario: se toma de la sesión autenticada, lo que impide suplantaciones.

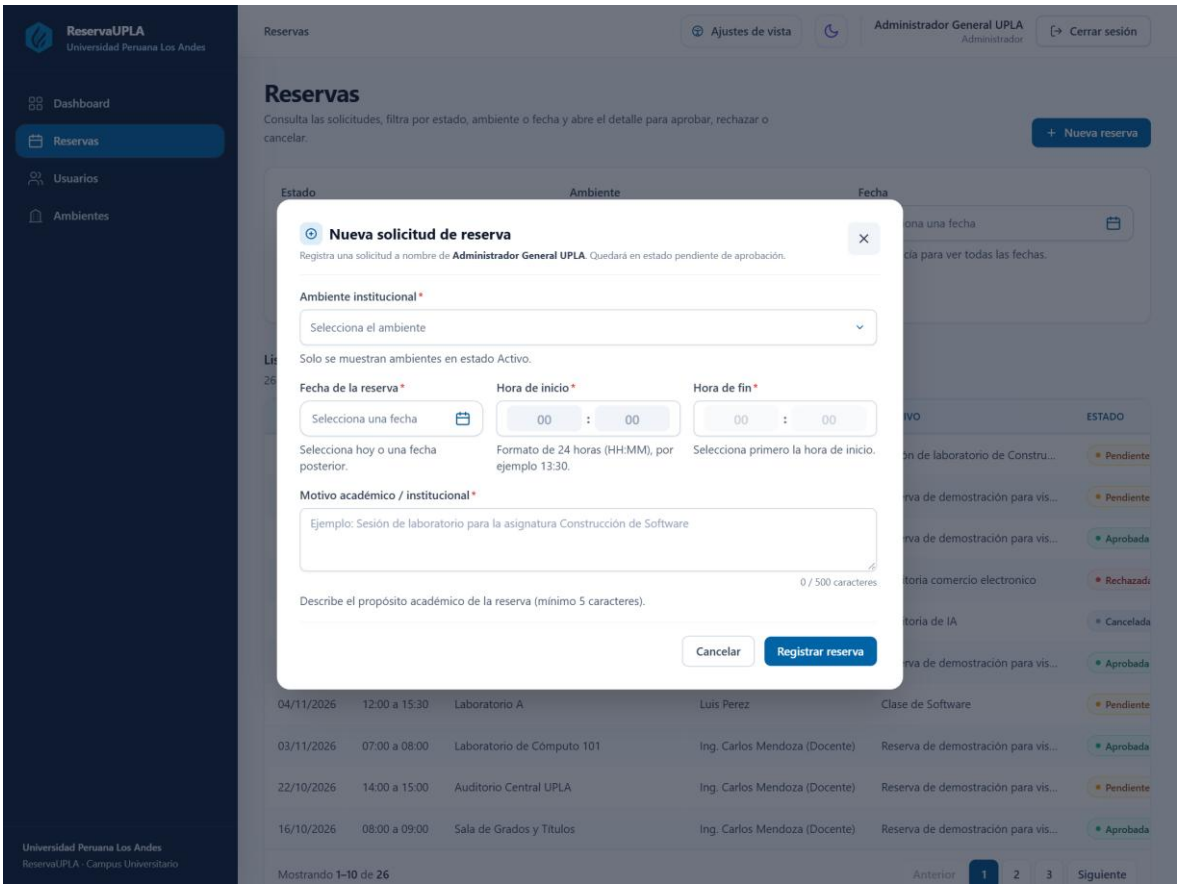


Figura 19. Diálogo de registro de una nueva solicitud de reserva

6.4. Formulario de usuarios

La gestión de usuarios está reservada al Administrador. El formulario captura nombre completo, correo institucional, rol y contraseña temporal. La contraseña debe cumplir cinco requisitos —longitud mínima de ocho caracteres, mayúscula, minúscula, número y símbolo— que se muestran como una lista de verificación que cambia de estado mientras el usuario escribe. Esta técnica reduce el error antes del envío y hace explícita la política, en lugar de revelarla mediante un mensaje tardío.

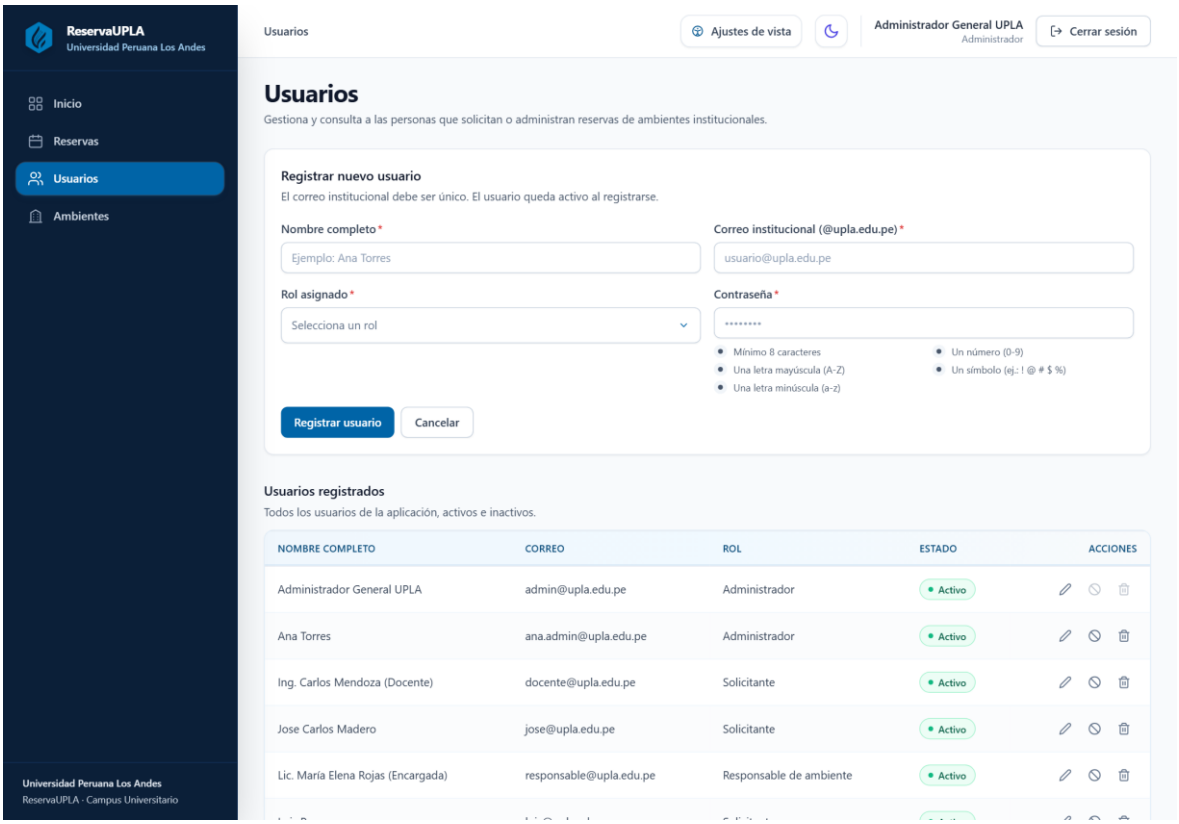


Figura 20. Formulario de registro de usuarios institucionales

El módulo de usuarios está reservado al Administrador y exige nombre, correo institucional único, rol y contraseña. La tabla inferior muestra los ocho usuarios registrados con su rol y estado, evidencia de que el alta se refleja de inmediato en el sistema.

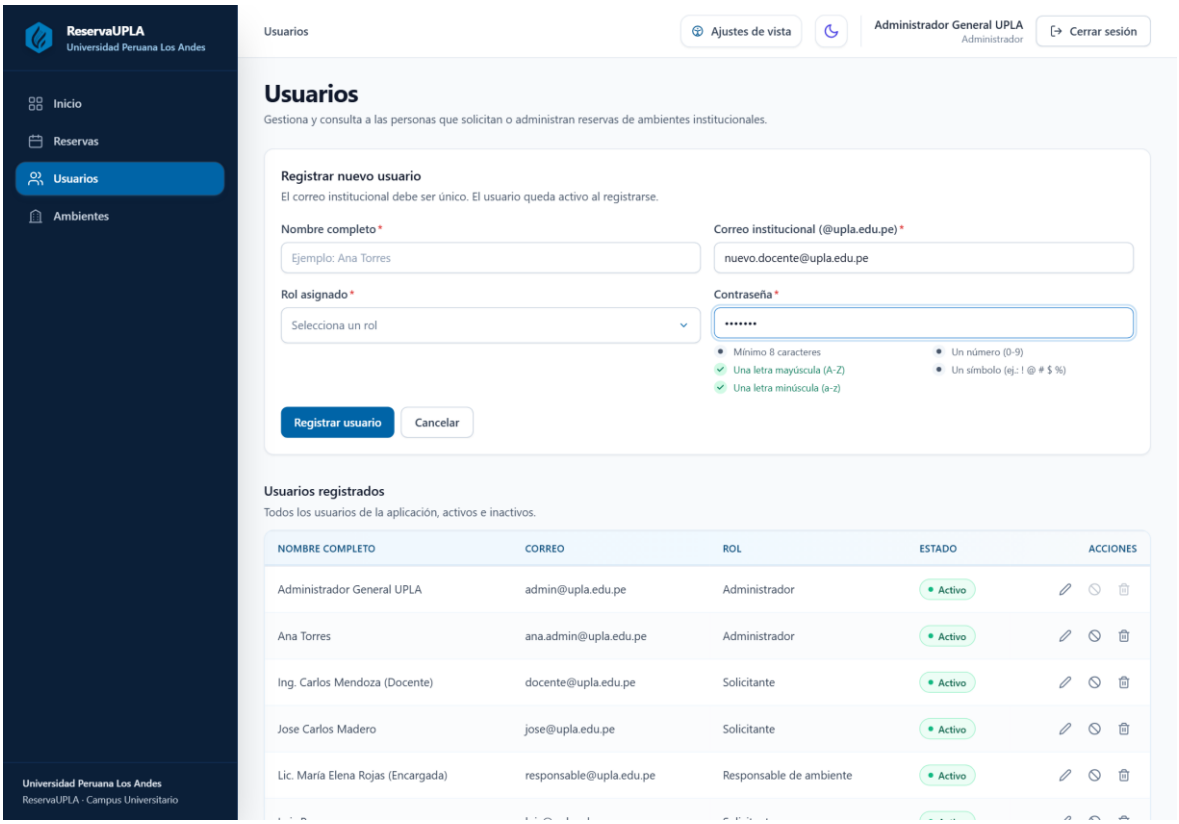


Figura 21. Política de contraseña con verificación en tiempo real

Los cinco requisitos se muestran como una lista que cambia de estado mientras el usuario escribe; los cumplidos aparecen en verde y los pendientes en gris. La misma política se aplica en el servidor, de modo que no puede eludirse construyendo la petición fuera del navegador.

6.5. Campos, obligatoriedad y destino de los datos

Tabla 21. Campos capturados por los formularios y su validación

Formulario	Campos principales	Obligatorios	Validación aplicada
Acceso	Correo, contraseña	Todos	Formato de correo, credenciales contra BCrypt
Usuarios	Nombre, correo, rol, contraseña	Todos	Longitud, correo, rol válido, política de contraseña
Ambientes	Nombre, tipo, capacidad, ubicación, estado	Todos	Nombre único, capacidad 1–5000, ubicación 3–100
Reservas	Ambiente, fecha, inicio, fin, motivo	Todos	Fecha no pasada, rango 07:00–22:00, fin > inicio, motivo 5–500

La captura de datos no se limita a recoger valores: cada campo declara su tipo, su rango y su mensaje de error. Esa declaración se comparte entre la interfaz y el servidor, aunque con implementaciones distintas, porque el frontend necesita reaccionar de inmediato y el backend necesita ser la autoridad final. En la sección siguiente se detalla cómo se articulan ambas barreras.

7. SEMANA 06: VALIDACIÓN Y CAPTURA DE DATOS

7.1. Estrategia de doble barrera

La validación se implementó en dos niveles deliberadamente independientes. En el **frontend**, las funciones de `validation.js` evalúan cada campo al perder el foco o al modificar su valor y muestran un mensaje junto al control; el objetivo es evitar que el usuario llegue al envío con un error previsible. En el **backend**, los DTO se validan con anotaciones de datos y, sobre todo, el servicio de aplicación vuelve a comprobar las reglas de negocio antes de persistir; el objetivo es que ninguna petición —aunque se construya fuera de la interfaz— pueda vulnerar una regla institucional.

Esta duplicación es intencional y no constituye una violación del principio DRY: se trata de dos responsabilidades distintas. La interfaz valida para **guiar**; el servidor valida para **proteger**. La primera puede ser evadida por cualquier cliente HTTP; la segunda es la que garantiza la integridad del dato almacenado. El sistema asume explícitamente que el navegador no es confiable.

McConnell (2004) sostiene que la validación es la primera línea de defensa contra errores, y Pressman y Maxim (2020) recuerdan que la calidad del dato de entrada depende de su exactitud, completitud y consistencia. Sommerville (2019) añade una condición de usabilidad: el mensaje de error debe explicar qué ocurrió y cómo corregirlo, nunca limitarse a un código opaco. Por eso el sistema no devuelve «Error 500» ni «EST=1», sino textos como «La hora de fin debe ser posterior a la de inicio (08:00)».

7.2. Implementación de las reglas de negocio RN-01 a RN-06

La siguiente matriz relaciona cada regla con su implementación en la interfaz, su control en el servidor y el código HTTP con el que se comunica el rechazo. Todas las reglas cuentan con evidencia ejecutada sobre el sistema.

Tabla 22. Matriz de reglas de negocio, doble validación y respuesta HTTP

Código	Regla	Frontend	Backend	HTTP
RN-01	No permitir más de una reserva aprobada para el mismo ambiente, fecha y horario	Mensaje en línea con el rango ocupado antes de enviar	`ObtenerTraslapeAprobadoAsync` con rangos semiabiertos; se revalida al aprobar	409
RN-02	Toda reserva registra fecha, hora de inicio, hora de fin, ambiente, solicitante y motivo	Campos obligatorios marcados con asterisco y validación por campo	DTO con `[Required]`; el solicitante se toma de la sesión y el estado del servidor	400
RN-03	La hora de fin debe ser mayor que la hora de inicio	`min` dinámico y mensaje específico con la hora de inicio elegida	`ValidarHorario` lanza excepción si `horaFin <= horaInicio`	400
RN-04	Las reservas solo pueden realizarse entre las 07:00 y las 22:00	Selectores acotados y validación de formato de 24 horas	`ValidarHorario` compara contra `HoraApertura` y `HoraCierre`	400
RN-05	Solo el administrador puede aprobar o rechazar una reserva	Botones ocultos para roles distintos de Administrador	Se recupera el actor desde la base y se verifica su rol activo	403
RN-06	Una reserva rechazada no puede modificarse	Se ocultan las acciones y se informa el estado final	`CambiarEstadoAsync` rechaza cambios sobre Rechazada y Cancelada	409

Dos decisiones merecen énfasis. La primera es que RN-01 se revalida al aprobar, no solo al crear. Si dos solicitudes pendientes se registran para el mismo horario —lo que es posible porque el traslape solo se controla entre reservas aprobadas—, la primera aprobación ocupa la franja y la segunda es rechazada con el mensaje que indica el horario en conflicto. La segunda es que el estado nunca se acepta desde el cliente: al crear, el servidor impone `Pendiente`; al cambiar de estado, valida la transición contra una máquina de estados explícita.

El control de traslape utiliza intervalos semiabiertos: existe conflicto si la reserva existente empieza antes de que termine la nueva y termina después de que empieza la nueva. Esta formulación permite que una reserva de 08:00 a 10:00 y otra de 10:00 a 12:00 convivan sin conflicto, y al mismo tiempo detecta cualquier cruce parcial. Es la interpretación correcta para franjas horarias contiguas y evita el error frecuente de comparar únicamente horas de inicio.

Myers, Sandler y Badgett (2011) definen la prueba como el proceso de ejecutar un programa con la intención de encontrar un defecto. Bajo esa definición, cada captura de rechazo de este informe es una prueba: no muestra que el sistema funciona, sino que se comporta correctamente cuando el usuario intenta algo inválido. La matriz RN-01 a RN-06 se construyó con ese criterio y por eso cada regla cuenta con evidencia de su rechazo, no solo de su cumplimiento.

7.3. Validaciones de frontend y mensajes al usuario

Los mensajes se redactaron para explicar qué ocurrió y cómo corregirlo, no para señalar culpa. La tabla siguiente reproduce las validaciones centralizadas y el texto exacto que recibe el usuario.

Tabla 23. Validaciones de frontend y mensajes asociados

Campo	Regla	Mensaje mostrado
Ambiente	Selección obligatoria	Selecciona el ambiente institucional.
Fecha	No anterior al día actual	No se pueden registrar reservas para fechas pasadas.
Hora de inicio	Formato HH:MM y rango 07:00–22:00	Ingresa una hora válida entre 07:00 y 22:00, en formato de 24 horas (HH:MM).
Hora de fin	Posterior a la hora de inicio	La hora de fin debe ser posterior a la de inicio (08:00).
Motivo	Entre 5 y 500 caracteres	El motivo debe tener al menos 5 caracteres descriptivos.
Nombre de ambiente	Entre 3 y 100 caracteres	El nombre debe tener al menos 3 caracteres.
Capacidad	Entero entre 1 y 5000	La capacidad debe ser un número entero entre 1 y 5000 personas.
Correo	Formato de correo institucional	Ingresa un correo electrónico con formato válido (@upla.edu.pe).
Contraseña	Cinco requisitos de seguridad	La contraseña debe incluir una letra mayúscula (A-Z).

7.4. Tratamiento uniforme de errores en el backend

Las excepciones de negocio se traducen a respuestas JSON con una estructura común: un mensaje legible, un código y, cuando corresponde, un diccionario de errores por campo. El middleware global evita repetir bloques `try/catch` en cada controlador y garantiza que el cliente reciba siempre el mismo formato, lo que simplifica el manejo en el frontend.

Tabla 24. Excepciones de negocio y su traducción a respuestas HTTP *Fuente: elaboración propia.*

Excepción	Situación	Código	Respuesta
BusinessRuleException (400)	Datos inválidos o incompletos	400	Mensaje y errores por campo
BusinessRuleException (409)	Conflicto de estado o de horario	409	Mensaje accionable con el rango ocupado
NotFoundException	Recurso inexistente	404	Mensaje genérico de no encontrado
ForbiddenException	Rol sin permiso para la operación	403	Mensaje de acceso denegado
Excepción no controlada	Fallo técnico inesperado	500	Mensaje genérico; detalle en el log

7.5. Evidencia de las validaciones

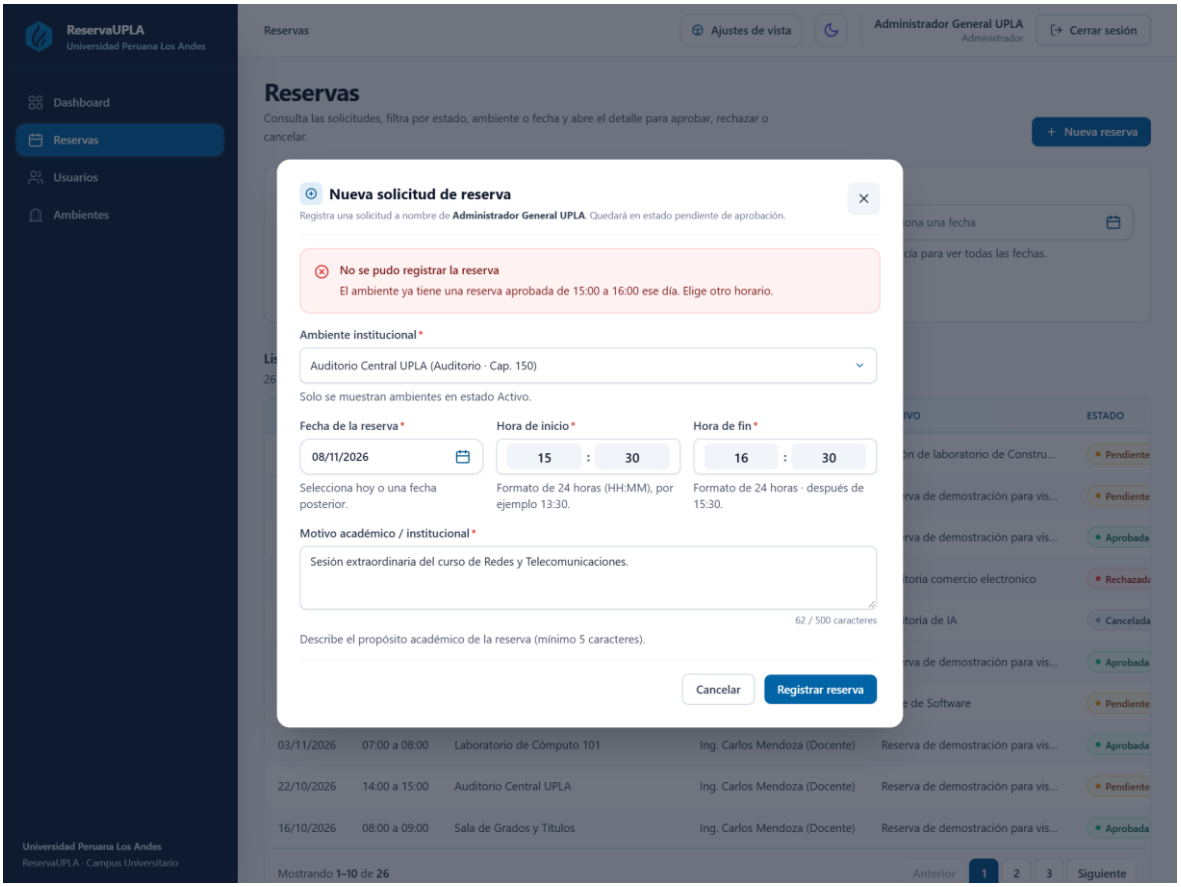


Figura 22. Rechazo de una reserva por traslape con un horario aprobado (RN-01)

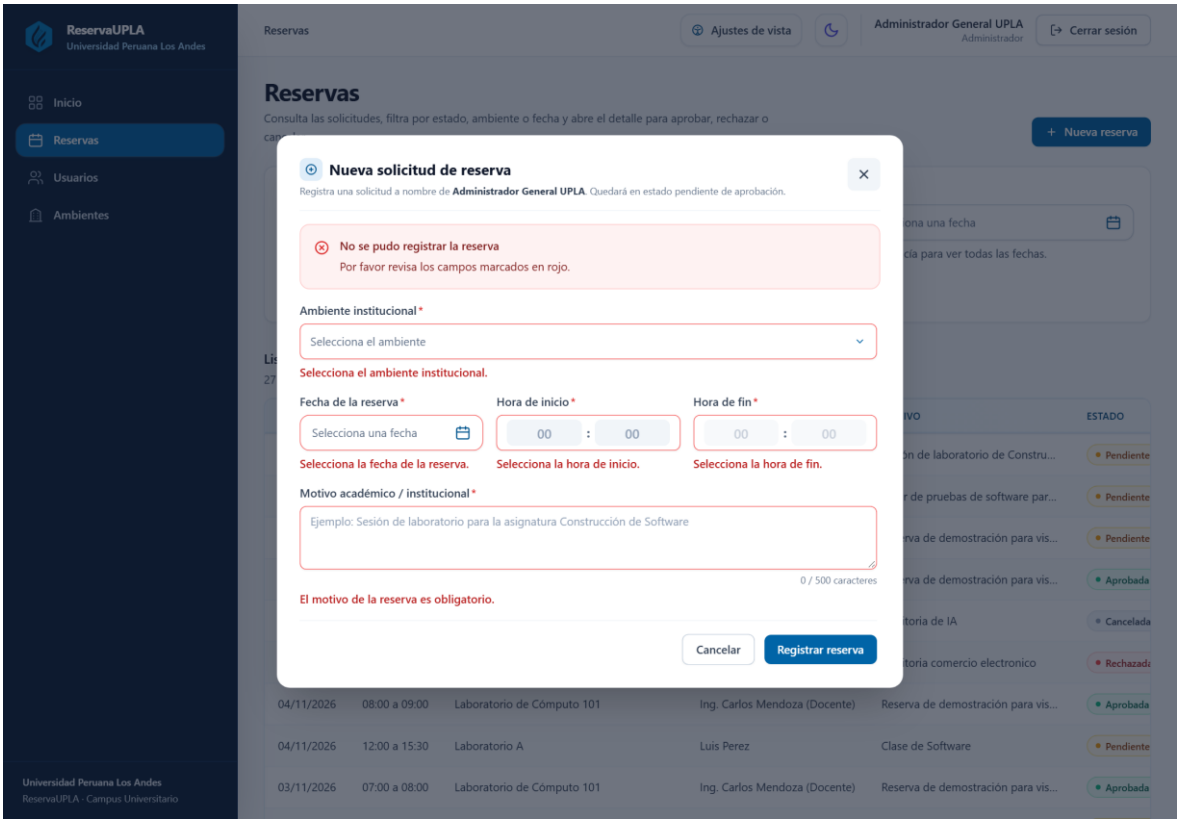


Figura 23. Validación de campos obligatorios de la reserva (RN-02)

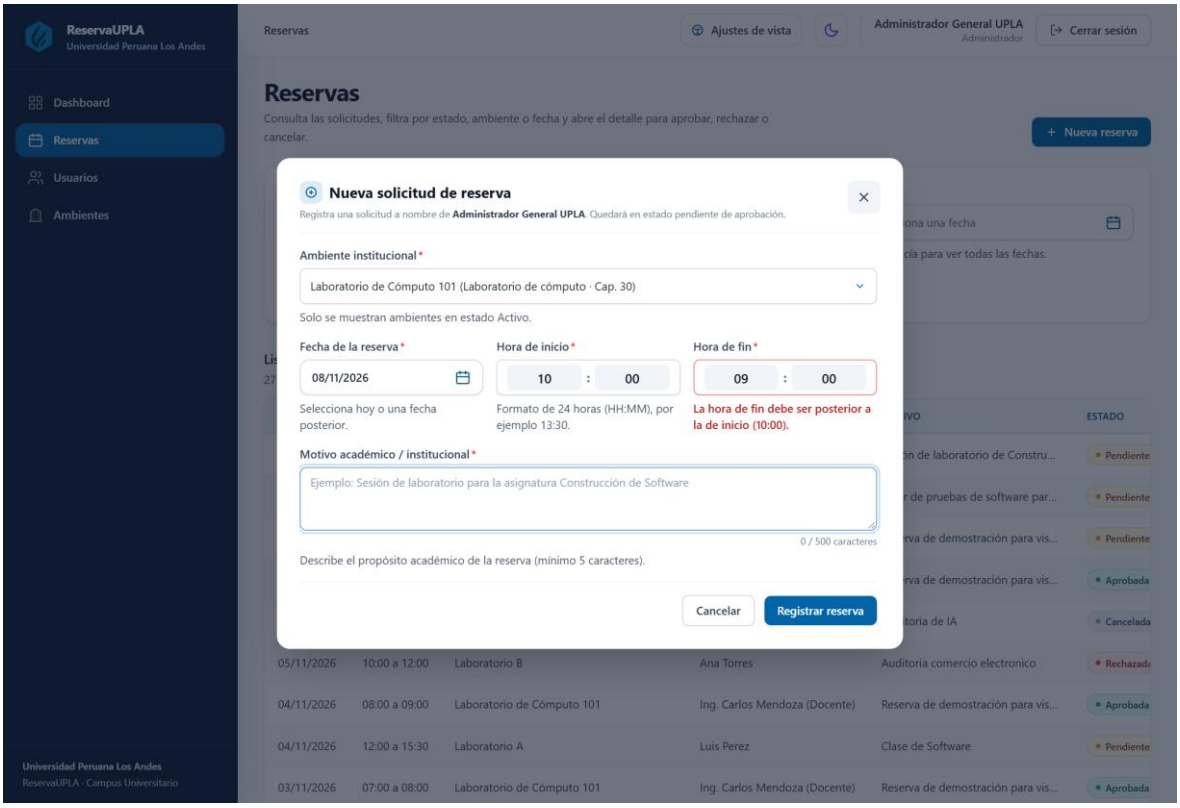


Figura 24. Validación de la hora de fin respecto de la hora de inicio (RN-03)

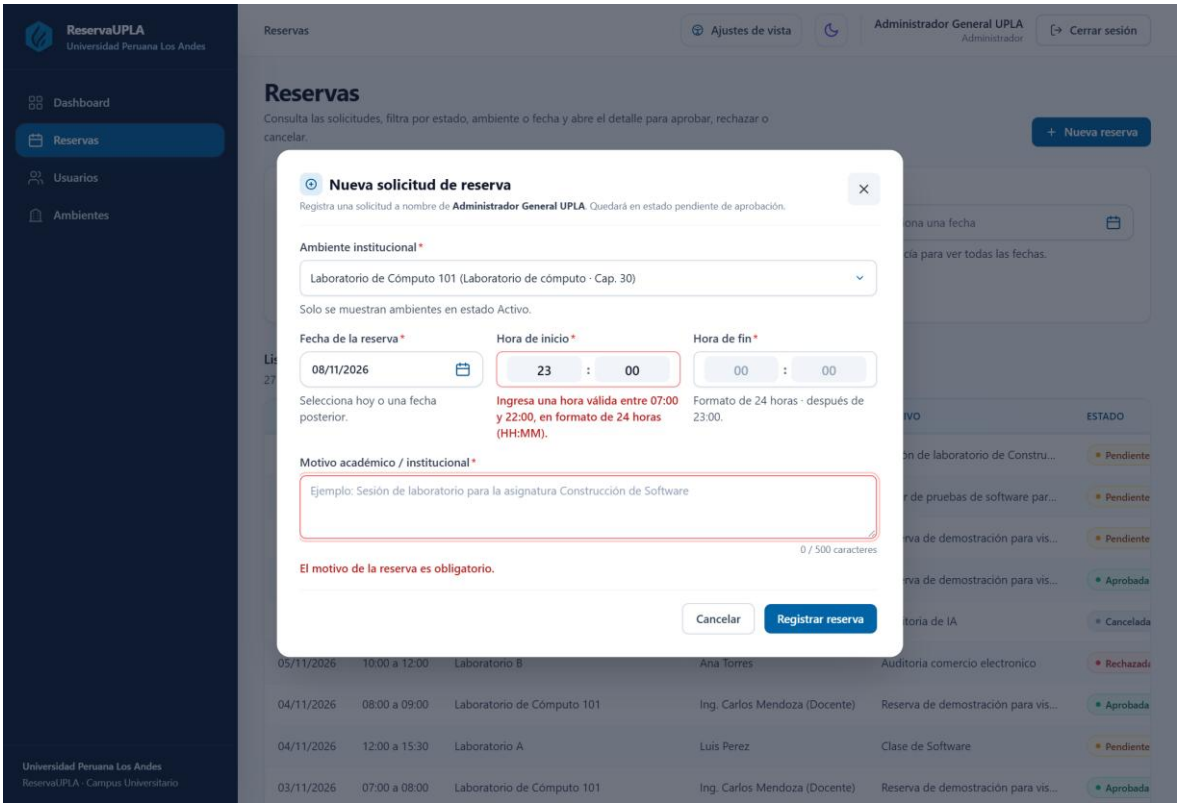


Figura 25. Restricción del horario universitario de 07:00 a 22:00 (RN-04)

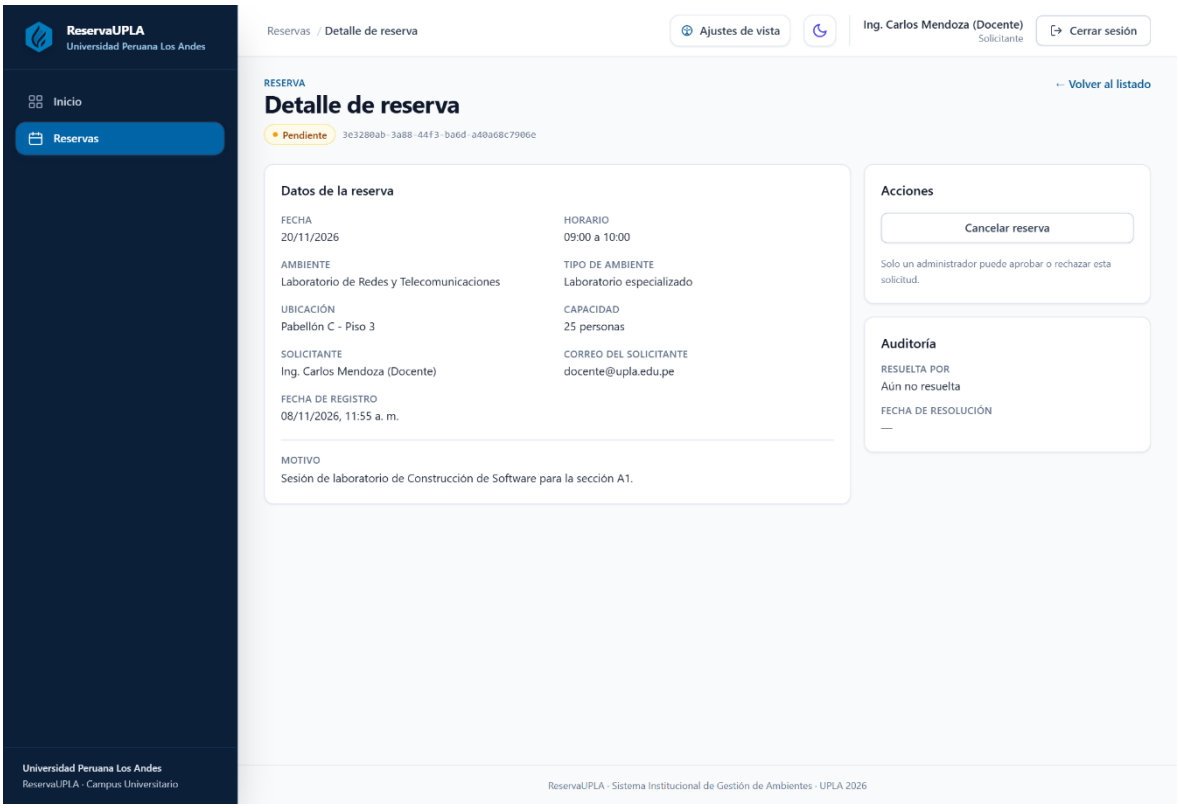


Figura 26. Detalle de reserva sin acciones de aprobación para el rol Solicitante (RN-05)

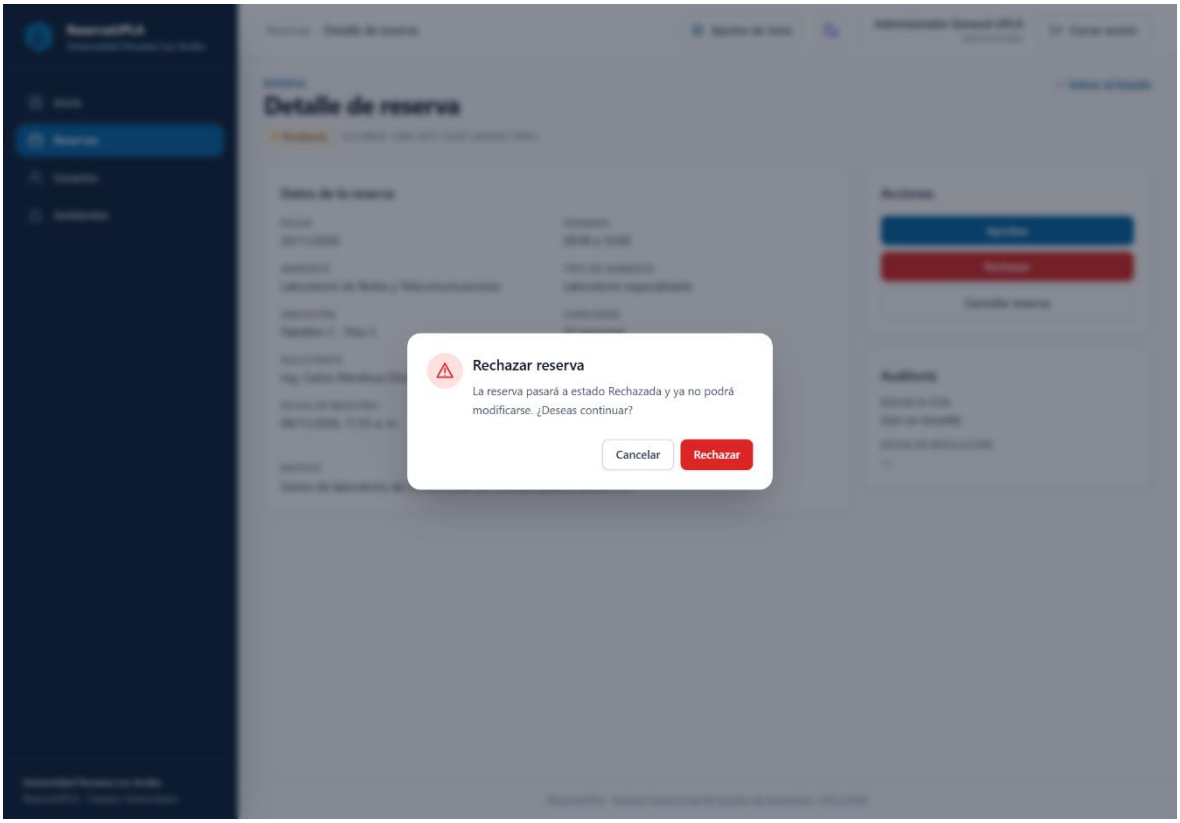


Figura 27. Diálogo de confirmación para rechazar una reserva (RN-05)

El diálogo advierte que la reserva pasará a estado Rechazada y que ya no podrá modificarse. El foco se coloca en «Cancelar», la tecla Escape cierra sin ejecutar y la confirmación es explícita, tres medidas coherentes con la irreversibilidad de la decisión.

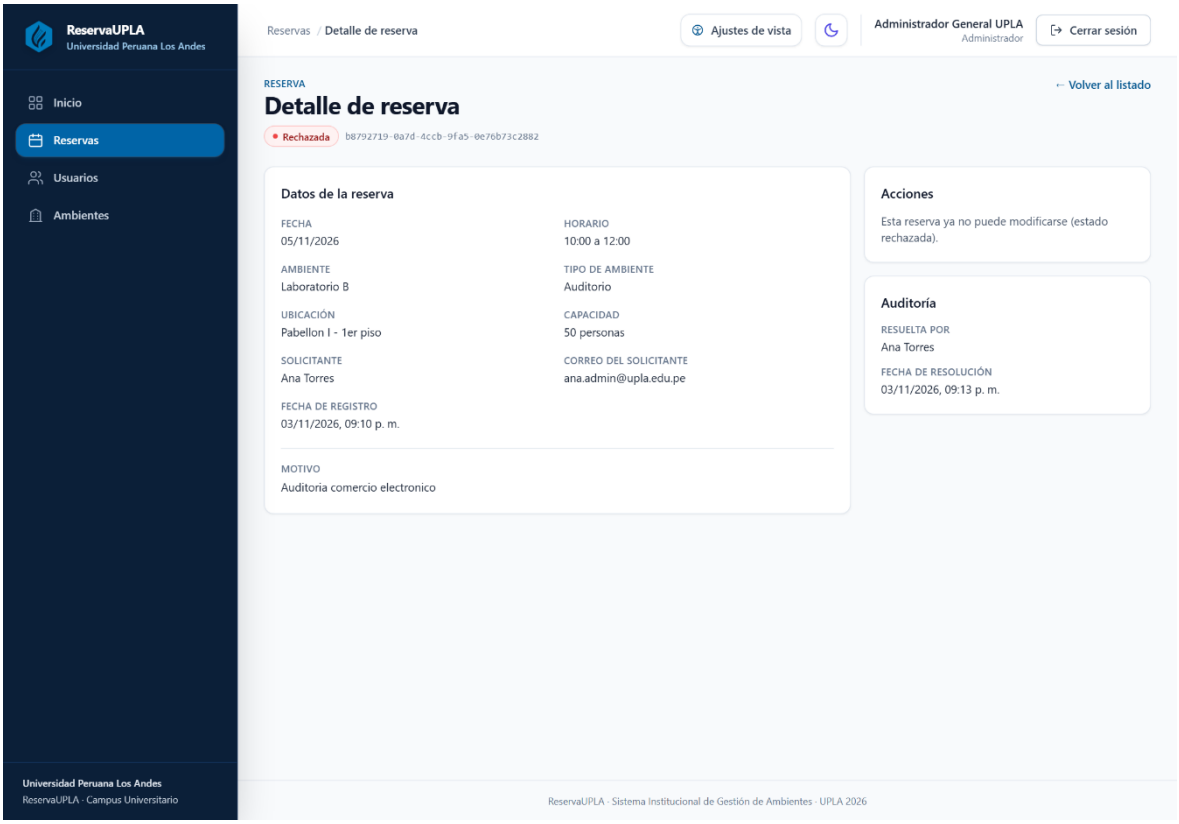


Figura 28. Reserva rechazada: acciones bloqueadas y auditoría de la decisión (RN-06)

La vista final confirma la regla RN-06: el sistema informa que la reserva ya no puede modificarse, retira todos los botones de acción y conserva el registro de auditoría con la persona que resolvió y la fecha exacta. La inmutabilidad no depende del rol del observador, sino del estado del registro.

Las seis capturas anteriores verifican el comportamiento en los dos niveles. Las dos primeras muestran el rechazo por traslape y por un rango horario inválido; las dos siguientes, la restricción de la jornada universitaria y la ausencia de acciones de aprobación para el rol Solicitante. Las dos últimas documentan el diálogo de rechazo que solo el Administrador puede invocar y el estado inmutable de una reserva rechazada, con el actor y la fecha de resolución como evidencia de auditoría. La verificación en el servidor se completó con invocaciones directas a la API, que devolvieron los códigos 409, 403 y 400 descritos en la matriz.

7.6. Trazabilidad entre requerimientos y reglas

Tabla 25. Trazabilidad entre requerimientos funcionales y reglas de negocio

Requerimiento	Reglas que lo protegen	Punto de control
RF-01 Registrar usuarios	Política de contraseña y correo único	`UsuarioService` y `PasswordPolicy`
RF-02 Registrar ambientes	RN-02 y unicidad de nombre	`AmbienteService`
RF-03 Registrar reservas	RN-01, RN-02, RN-03 y RN-04	`ReservaService.CrearAsync`
RF-04 Validar disponibilidad	RN-01	`ObtenerTraslapeAprobadoAsync`
RF-05 y RF-06 Consultar	Control de sesión	Middleware JWT y filtros de consulta
RF-07 Cambiar estado	RN-05 y RN-06	`ReservaService.CambiarEstadoAsync`
RF-08 Mostrar indicadores	Consistencia de estados	`DashboardService` y consultas agregadas

8. SEMANA 07: REUTILIZACIÓN DE CÓDIGO

8.1. Estrategia de componentes

La reutilización se abordó desde dos frentes. En el backend, las interfaces de repositorio y servicio permiten sustituir implementaciones sin modificar los casos de uso, y las excepciones comunes evitan duplicar el tratamiento de errores. En el frontend, se construyó un catálogo de componentes de presentación que concentran comportamiento, accesibilidad y estilo; las páginas los componen y no reimplementan controles.

Cada componente tiene una responsabilidad única, recibe sus datos por propiedades y comunica eventos mediante callbacks. Ninguno conoce la API ni el usuario autenticado: esa información permanece en las páginas y en los servicios. Esta separación facilita las pruebas, permite cambiar la presentación sin tocar la lógica y evita el acoplamiento que aparece cuando un campo de formulario conoce el endpoint al que enviará sus datos.

Fowler (2018) advierte que la duplicación es el olor de código más costoso, porque cada copia evoluciona por separado y produce inconsistencias. La reutilización de componentes en ReservaUPLA ataca ese riesgo: un campo, un aviso o un diálogo se definen una vez y se configuran en todos los módulos. El resultado no es solo menos código, sino un comportamiento uniforme que el usuario aprende una sola vez.

8.2. Catálogo de componentes mínimos

Los nueve componentes sugeridos por el caso práctico se implementaron y se integraron en más de un módulo. La columna de reutilización indica la cantidad de archivos que importan el componente, dato obtenido del propio repositorio.

Tabla 26. Catálogo de componentes reutilizables y su nivel de uso

Nº	Componente	Archivo	Propósito	Usos
1	TextInput	`TextInput.jsx`	Campo de texto con etiqueta, ayuda y error	6
2	SelectInput	`SelectInput.jsx`	Selector con opciones, placeholder y estado de carga	7
3	DateInput	`DateInput.jsx`	Selector de fecha con límites y formato local	3
4	TimeInput	`TimeInput.jsx`	Selector horario de 24 horas acotado a la jornada	2
5	Button	`Button.jsx`	Botón con variantes, tamaño y estado de envío	11
6	AlertMessage	`AlertMessage.jsx`	Aviso con variantes de éxito, error, advertencia e información	10
7	ModalConfirmacion	`ModalConfirmacion.jsx`	Diálogo accesible con trampa de foco y foco seguro	4
8	TableList	`TableList.jsx`	Grilla con columnas configurables y estado vacío	5
9	CardIndicador	`CardIndicador.jsx`	Tarjeta métrica con tono, icono y esqueleto de carga	2

Además de los nueve mínimos, el proyecto incorporó componentes de apoyo que también se reutilizan: `FieldWrapper` (5 usos) unifica etiqueta, texto de ayuda y mensaje de error; `EstadoBadge` (4 usos) traduce el estado a color y texto; `Spinner` (5 usos) indica carga; `Breadcrumbs` (1 uso) genera la ruta de navegación desde la URL; `PasswordRequirements` (2 usos) muestra la lista de verificación de la contraseña; y `TextAreaInput` (1 uso) especializa el campo de texto largo.

8.3. Evidencia de los componentes en uso

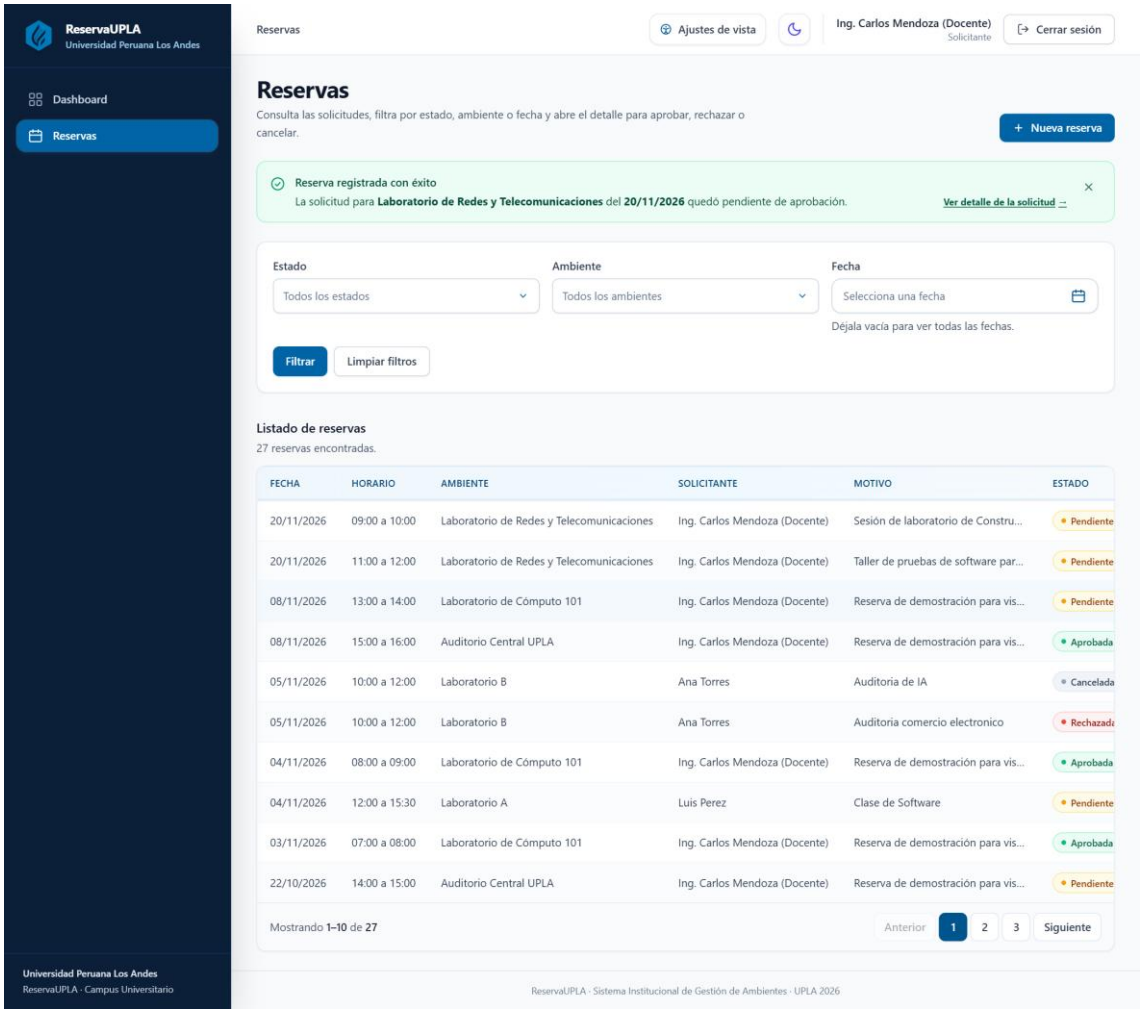


Figura 29. Variantes de AlertMessage y badges de estado en el listado

La Figura 28 muestra cómo un mismo componente de aviso se adapta a contextos distintos sin reescribir estilos ni estructura. Los badges, por su parte, traducen un valor de dominio —Pendiente, Aprobada, Rechazada o Cancelada— a una señal visual consistente. Si se agregara un nuevo estado, bastaría con ampliar la configuración del componente; las páginas no tendrían que modificarse.

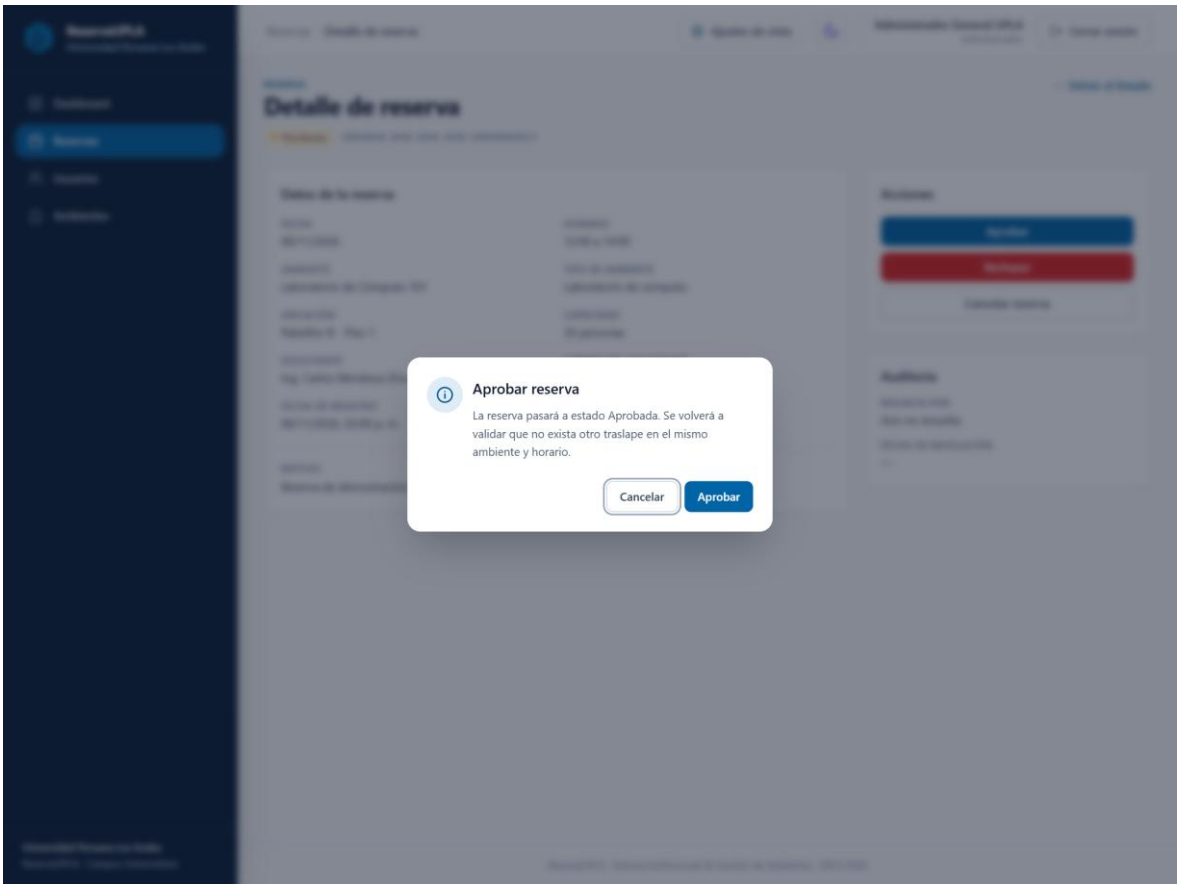


Figura 30. Diálogo de confirmación con foco inicial en la acción segura

El diálogo de confirmación implementa tres medidas de prevención de errores: el foco se coloca en «Cancelar» y no en la acción destructiva, la tecla Escape cierra el diálogo sin ejecutar cambios y el foco queda atrapado mientras el diálogo permanece abierto. Estas medidas son especialmente importantes en operaciones de rechazo o cancelación, donde un clic accidental tendría consecuencias difíciles de revertir.

8.4. Beneficios observados

- **Consistencia visual y de comportamiento.** Los formularios comparten altura de campo, espaciado, color de error y posición del mensaje, lo que reduce la carga cognitiva y los errores de interpretación.
- **Accesibilidad centralizada.** Las etiquetas `aria`, el foco visible, la trampa de foco y los anuncios con `aria-live` se implementaron una vez y benefician a todos los módulos.
- **Menor superficie de mantenimiento.** Un cambio en el comportamiento de un botón o de un aviso se aplica en un archivo y se propaga sin editar cada página.
- **Vocabulario común.** El equipo nombra las mismas cosas del mismo modo en la interfaz, el código y la documentación, lo que facilita la incorporación de nuevos desarrolladores.

9. SEMANA 08: REFINAMIENTO DE CÓDIGO Y DEUDA TÉCNICA

El refinamiento se orientó a eliminar duplicación, reducir el acoplamiento y mejorar la legibilidad sin alterar el comportamiento observable. Las intervenciones se registraron como tareas de refactorización y se verificaron contra los mismos escenarios funcionales ya validados.

9.1. Refactorizaciones aplicadas

Tabla 27. Refactorizaciones ejecutadas y resultado

Hallazgo	Acción realizada	Resultado
Vista independiente de creación de reservas duplicaba el formulario	Se eliminó la página y se conservó el modal sobre el listado	Un solo flujo de creación y continuidad de contexto
Llamadas HTTP repetidas con manejo de token disperso	Se centralizó todo en `httpClient.js`	Inyección única de JWT y tratamiento uniforme de 401
Reglas de validación replicadas en varios formularios	Se concentraron en `utils/validation.js`	Mensajes y criterios coherentes
Colores y textos de estado repetidos en cada tabla	Se creó `EstadoBadge` con una configuración única	Cambio de estado centralizado
Migas de pan construidas manualmente por página	Se implementó `Breadcrumbs` a partir de la ruta	Navegación consistente y sin mantenimiento manual
Política de contraseña duplicada entre alta y edición	Se unificó en `PasswordRequirements` y `PasswordPolicy`	Una sola definición de la regla

McConnell (2004) describe la deuda técnica como el efecto acumulado de las decisiones rápidas: el software evoluciona y, sin disciplina técnica, el código se degrada progresivamente hasta que cada cambio cuesta más que el anterior. Las refactorizaciones de esta semana se eligieron por ese criterio: eliminar la vista duplicada de creación, centralizar el cliente HTTP y unificar las validaciones reduce el costo de los cambios futuros.

9.2. Mejoras de claridad y seguridad

Se revisaron los controladores para asegurar que no contuvieran lógica de decisión; se eliminaron variables y dependencias sin uso; se normalizaron los nombres de las respuestas; y se sustituyeron comparaciones de cadenas por constantes de dominio (`EstadoReserva`, `EstadoAmbiente`, `RolUsuario`). Este último cambio elimina los errores tipográficos que suelen aparecer cuando los estados se escriben como texto libre en distintos archivos.

En seguridad se verificó que la autorización no dependiera de la interfaz, que las contraseñas no se devolvieran en ningún DTO y que los mensajes de error no revelaran detalles internos. También se ajustó el contraste de textos secundarios y el comportamiento del botón de visibilidad de la contraseña, que interfería con el control nativo del navegador.

9.3. Deuda técnica identificada

Un informe de refinamiento honesto debe reconocer lo que queda pendiente. Las siguientes deudas se documentan con su impacto y su prioridad, no se ocultan ni se presentan como fortalezas.

Tabla 28. Deuda técnica pendiente y prioridad de atención

Deuda	Impacto	Prioridad
Ausencia de pruebas automatizadas de integración	Las regresiones se detectan con verificación manual	Alta
Cálculo de gráficos en el cliente sobre el listado completo	El rendimiento se degrada con volúmenes altos	Media
Listados sin paginación en el servidor	Respuestas grandes y tiempos de carga mayores	Media
Sin rotación ni revocación de tokens	Una sesión robada permanece válida hasta su expiración	Media
Sin límite de intentos en el inicio de sesión	Ventana para ataques de fuerza bruta	Alta
Sin notificaciones automáticas al solicitante	El seguimiento depende de la consulta manual	Baja

La priorización responde a riesgo, no a esfuerzo. Las dos deudas de prioridad alta —pruebas automatizadas y límite de intentos— afectan directamente la confiabilidad y la seguridad; las de prioridad media condicionan la escalabilidad; la de prioridad baja mejora la experiencia pero no compromete la operación del prototipo. Todas ellas constituyen el punto de partida natural de la siguiente iteración.

Martin (2018) afirma que el código limpio parece escrito por alguien a quien le importan los demás, porque anticipa el esfuerzo de quien lo leerá después. Esa idea guía la deuda declarada: las tareas pendientes se documentan con impacto y prioridad para que la siguiente iteración no empiece por descubrirlas, sino por atenderlas.

10. SEMANA 09: MÓDULOS DE SALIDA, PANEL DE INICIO Y AUDITORÍA

10.1. Indicadores del Panel de Inicio

El tablero presenta los cinco indicadores mínimos exigidos por el caso práctico. Los valores que se muestran a continuación corresponden al estado real de la base de datos al cierre del informe y coinciden con la salida de `psql` presentada en la sección 4.5.

Tabla 29. Indicadores del Panel de Inicio y valor verificado

Indicador	Valor	Cálculo	Interpretación
Total de reservas	27	COUNT(*) sobre Reservas	Incluye todos los estados, incluso canceladas
Reservas pendientes	5	COUNT con Estado = 'Pendiente'	Solicitudes en espera de decisión
Reservas aprobadas	18	COUNT con Estado = 'Aprobada'	Reservas que ocupan el ambiente
Reservas rechazadas	3	COUNT con Estado = 'Rechazada'	Solicitudes denegadas
Ambiente más utilizado	Laboratorio de Cómputo 101 · 8	Agrupación por ambiente sobre aprobadas	Espacio con mayor ocupación efectiva

El indicador de ambiente más utilizado se calcula únicamente sobre reservas aprobadas, porque son las que efectivamente ocuparon el espacio. Contar solicitudes pendientes o rechazadas distorsionaría la lectura de ocupación. El resultado se presenta como una insignia destacada dentro del gráfico de uso de ambientes, de modo que el dato principal y su contexto aparecen juntos.

Nota de diseño y adaptación por roles: Mientras que el Administrador y el Responsable de ambiente visualizan los cinco indicadores consolidados de todo el campus universitario (Tabla 29), el Panel de Inicio del Solicitante adapta automáticamente sus tarjetas para reflejar su actividad individual («Mis solicitudes», «Mis pendientes», «Mis aprobadas» y «Mis rechazadas»), garantizando que el docente reciba información operativa relevante para su labor sin exponer cifras globales ajenas a su función.

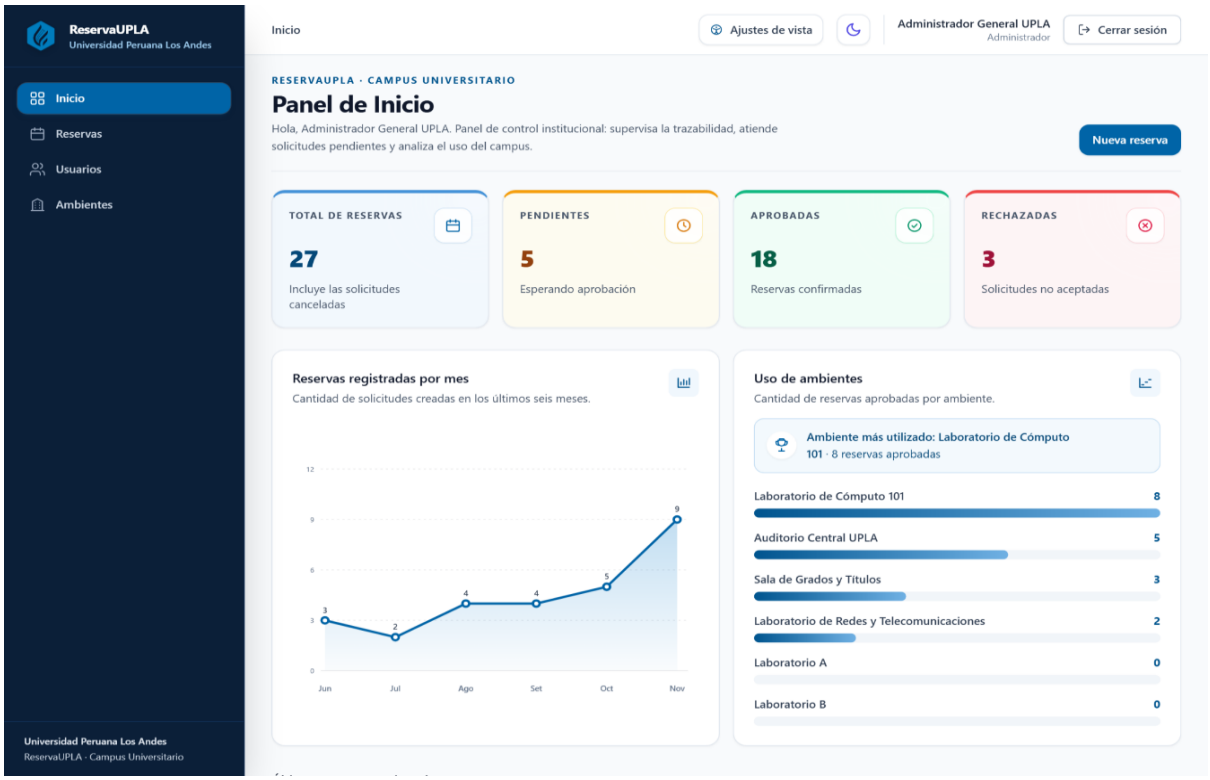


Figura 31. Panel de Inicio en modo claro con los cinco indicadores

El modo oscuro no es un adorno: demuestra que los colores se definieron como tokens semánticos y no como valores fijos en cada pantalla. Al alternar el tema, las tarjetas, tablas, gráficos y mensajes conservan su jerarquía y su contraste. La preferencia se guarda en el dispositivo, de modo que el usuario no debe configurarla en cada visita.



Figura 32. Panel de Inicio en modo oscuro Fuente: captura del sistema ReservaUPLA en ejecución.

Nielsen (2020) advierte que cada dato innecesario o mal rotulado aumenta la carga cognitiva y obliga al usuario a traducir el sistema a su propio lenguaje. Por eso el panel prohíbe los códigos crudos —nunca «EST=1» ni «PRI=A»— y presenta rótulos humanos como «Estado: Pendiente». La misma heurística, correspondencia entre el sistema y el mundo real, justifica que el módulo se rotule «Inicio» en lugar de «Dashboard»: el usuario institucional reconoce la palabra, no el anglicismo.

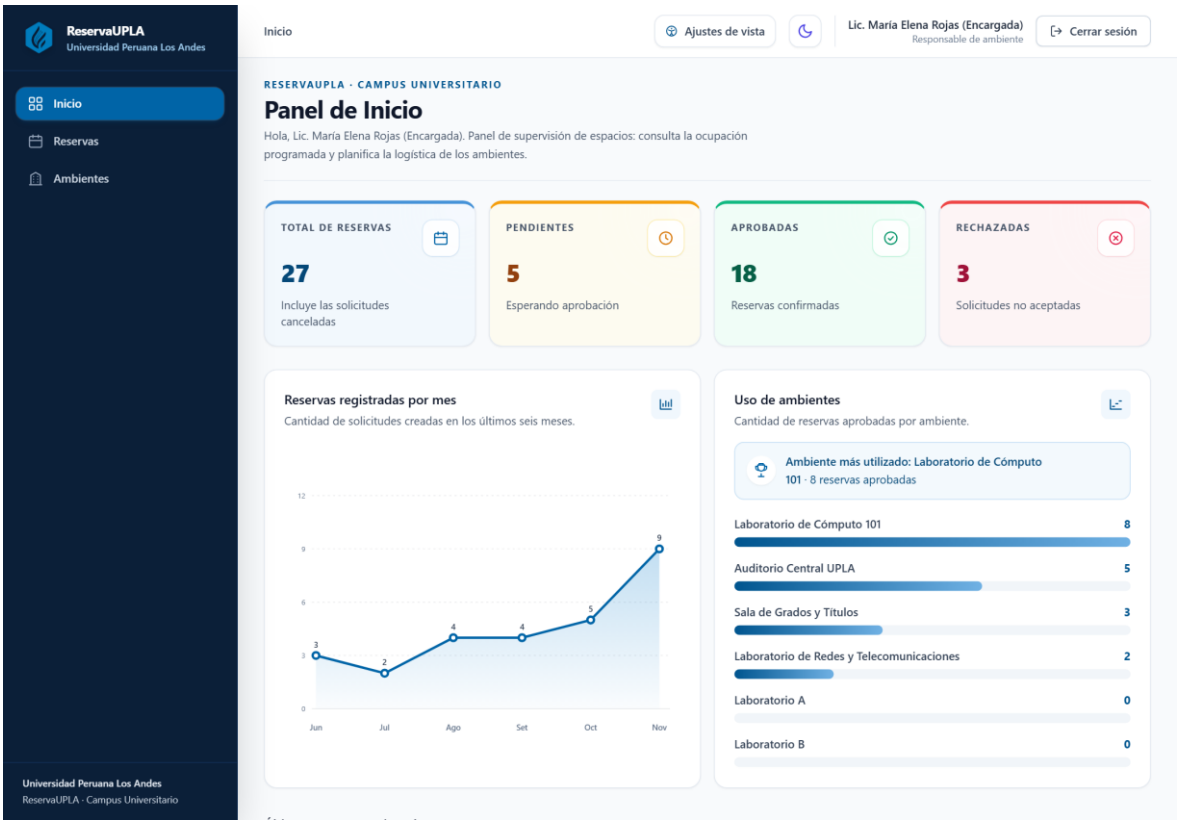


Figura 33. Panel de Inicio desde el rol Responsable de ambiente

La misma pantalla se adapta al rol: el encabezado cambia a «Panel de supervisión de espacios», el subtítulo invita a consultar la ocupación programada y el menú lateral omite el módulo Usuarios. El Responsable de ambiente utiliza esta vista para planificar la apertura de laboratorios y el control de llaves y equipamiento, sin acceder a la gestión de cuentas ni a la creación de reservas.

10.2. Visualizaciones de tendencia y ocupación

El primer gráfico muestra la cantidad de solicitudes registradas en los últimos seis meses; el segundo compara las reservas aprobadas por ambiente mediante barras horizontales. Ambos se construyeron con SVG nativo y sin bibliotecas externas de gráficos, lo que reduce el peso de la aplicación y permite controlar la accesibilidad: cada visualización expone una descripción textual con `role="img"` y `aria-label`, de modo que un lector de pantalla pueda anunciar los datos.

La serie mensual se calcula a partir de `FechaRegistro`, no de la fecha de uso, porque el indicador busca reflejar la actividad de solicitud. La comparación por ambiente, en cambio, se calcula sobre la fecha de uso y solo con reservas aprobadas. Esta distinción evita interpretaciones erróneas: una cosa es cuándo se pidió el espacio y otra cuándo se ocupó.

10.3. Listado de reservas y vistas por rol

El listado presenta fecha, horario, ambiente, solicitante, motivo, estado y una acción para consultar el detalle. Incorpora filtros combinables por estado, ambiente y fecha, que se envían al servidor como parámetros de consulta. La tabla reutiliza `TableList`, por lo que hereda el estado vacío, la alineación de columnas y el comportamiento responsive.

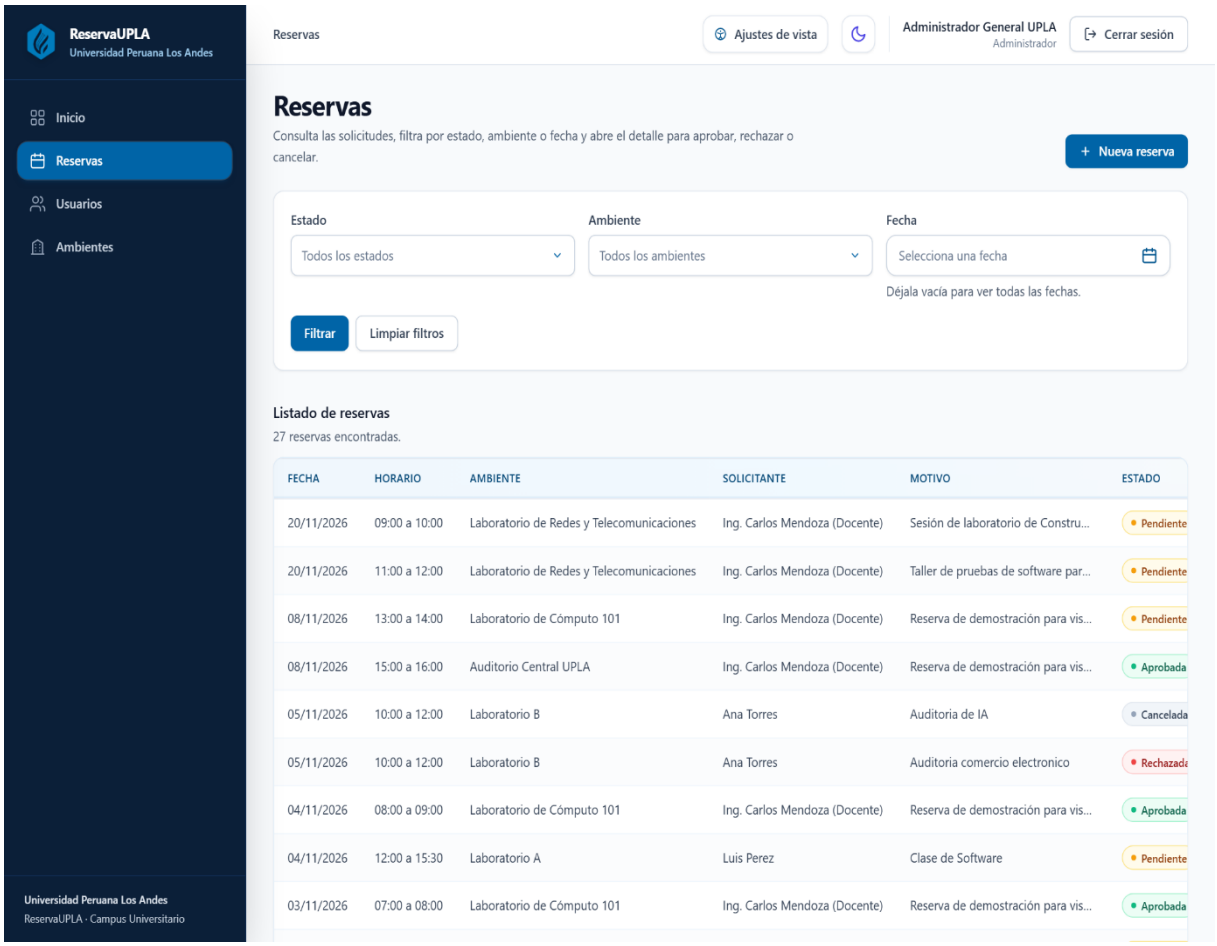


Figura 34. Listado de reservas con filtros y estados, vista del Administrador

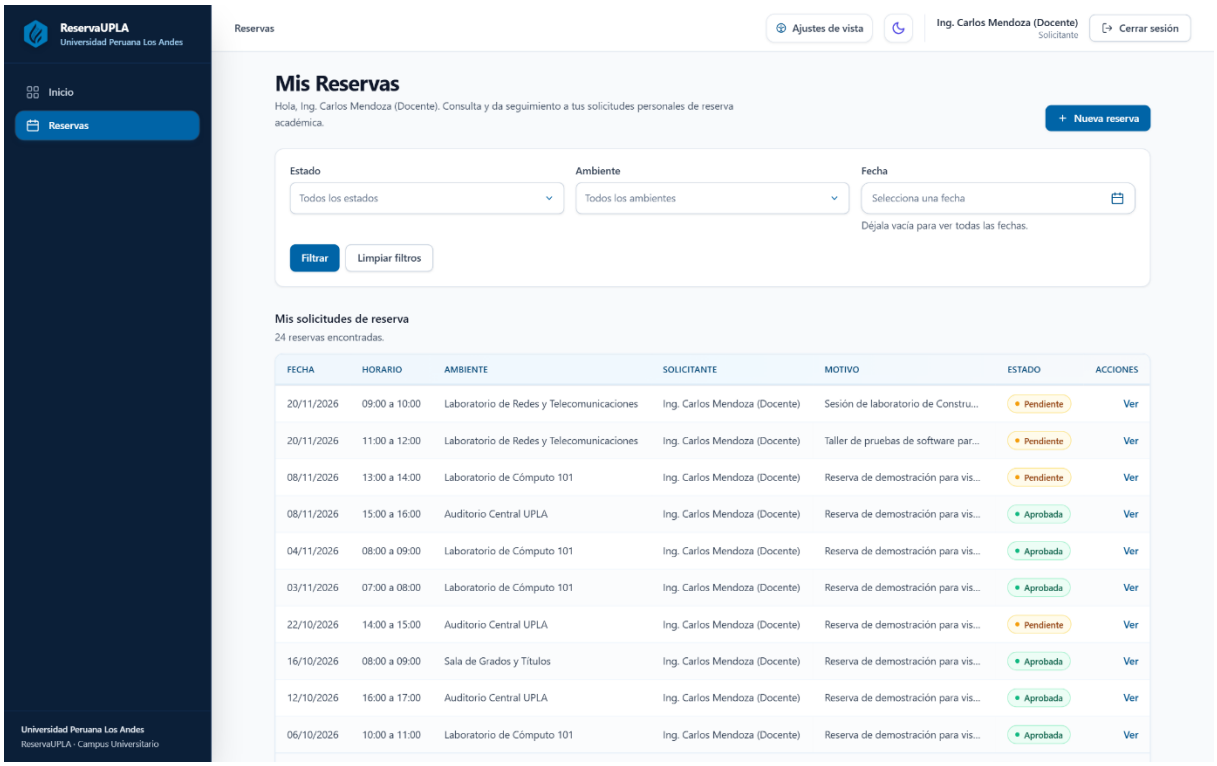


Figura 35. Vista personalizada «Mis Reservas» con filtrado de privacidad para el rol Solicitante

La comparación entre la **Figura 34** (Administrador) y la **Figura 35** (Solicitante) demuestra la aplicación rigurosa del **Principio de Menor Privilegio** y la **protección de la privacidad de la información institucional**:

1. **Vista del Administrador (Figura 34):** Presenta el catálogo consolidado de todas las solicitudes de la universidad (27 registros en estado activo o histórico). Este alcance es indispensable para que la autoridad universitaria evalúe la carga global, resuelva solicitudes pendientes (RN-05) y mantenga la trazabilidad de auditoría de todo el campus.
2. **Vista del Solicitante (Figura 35):** La interfaz se transforma dinámicamente en el módulo **«Mis Reservas»**. Tanto el frontend como los servicios de aplicación aíslan los datos para que el docente visualice **únicamente las solicitudes creadas bajo su propia cuenta**, ocultando las reservas y justificaciones académicas de otros colegas. Esta decisión mitiga el riesgo de exposición de datos sensibles entre pares, se alinea con la **Heurística 8 de Nielsen (diseño minimalista y reducción de la carga cognitiva)** y garantiza la **continuidad de contexto de Shneiderman**, ya que el docente gestiona su historial sin ruido visual innecesario.

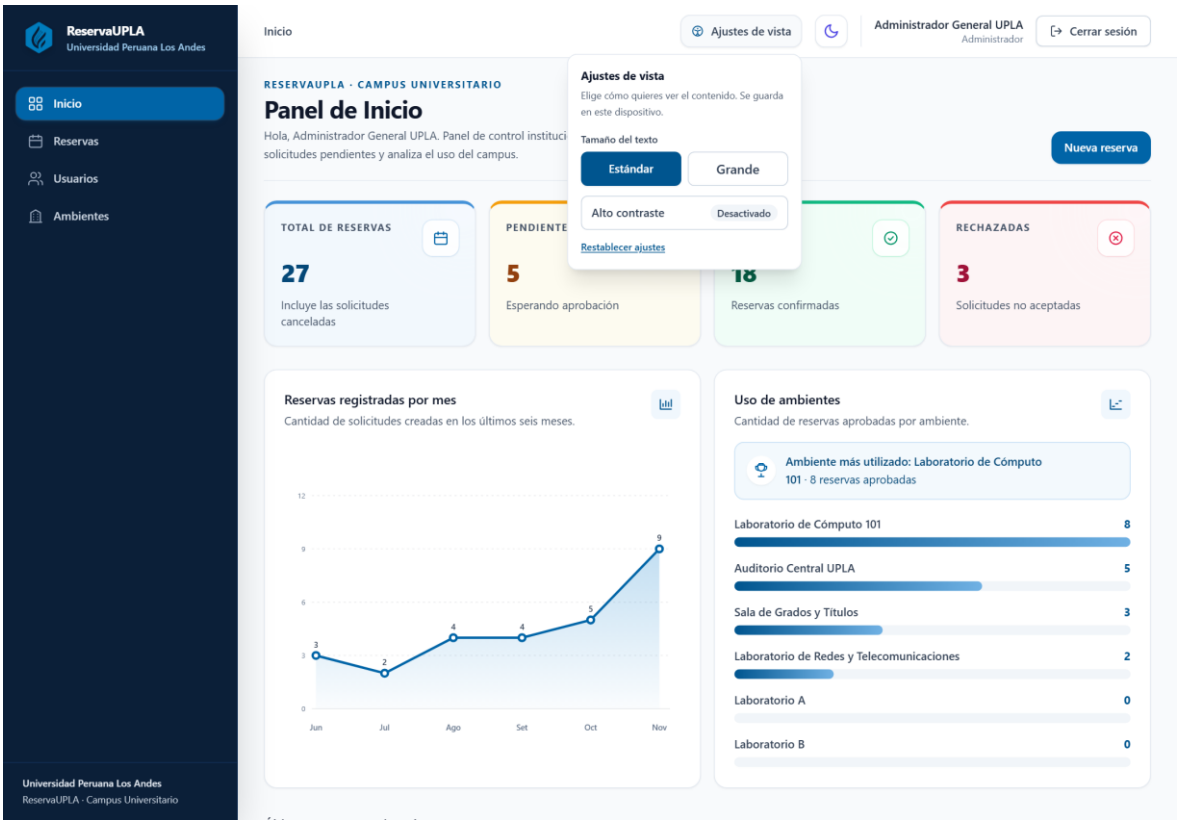


Figura 36. Panel de ajustes de accesibilidad y contraste.

El panel permite ampliar el texto, activar alto contraste y restablecer los valores iniciales, y conserva la preferencia en el dispositivo. Cumple los criterios WCAG de redimensionamiento y contraste, y responde a la heurística 7 de Nielsen al dejar el control en manos del usuario.

10.4. Detalle de la reserva y trazabilidad de auditoría

La vista de detalle reúne los datos de la reserva, la información del ambiente y del solicitante, el motivo completo y el panel de acciones. Cuando la reserva ya fue resuelta, muestra quién tomó la decisión y en qué momento; cuando sigue pendiente, habilita las acciones según el rol. Esta pantalla es la principal evidencia de trazabilidad del sistema.

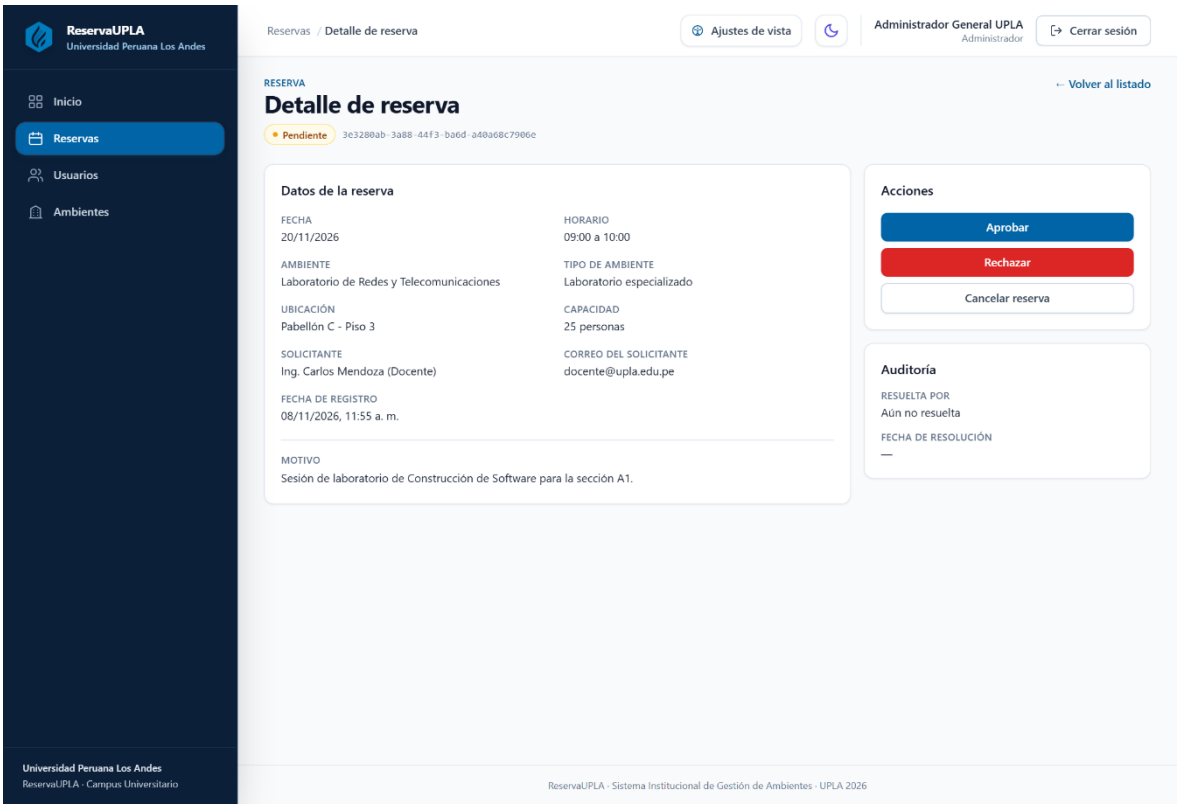


Figura 37. Detalle de reserva con registro de auditoría de la decisión

El registro de auditoría se compone de tres datos: el estado resultante, el identificador del usuario que resolvió y la marca temporal de la decisión. Estos campos se escriben en el servidor a partir del actor autenticado, nunca desde el cliente. En consecuencia, la afirmación «esta reserva fue aprobada por determinada persona en determinado momento» puede sostenerse con el dato almacenado y no con una suposición.

10.5. Consideraciones de rendimiento y consulta

Las consultas de listado utilizan `AsNoTracking` porque son de solo lectura, lo que evita el costo de rastreo de cambios. Las relaciones necesarias se cargan con `Include` para no incurrir en consultas adicionales por fila. La consulta de traslape se apoya en el índice compuesto por ambiente, fecha y estado. Finalmente, el dashboard combina una consulta agregada de conteos con el listado y el catálogo, y calcula las series en el cliente; esta decisión es adecuada para el volumen actual y se reconoce como deuda técnica para volúmenes mayores.

11. RESPUESTA A LA PREGUNTA DE REFLEXIÓN FINAL

> *Explique cómo la aplicación de estándares, validación de datos, reutilización de componentes y refinamiento de código contribuye a mejorar la calidad, mantenibilidad y evolución del sistema ReservaUPLA.*

La construcción de software profesional no consiste únicamente en lograr que un programa funcione, sino en asegurar que sea comprensible, verificable y sostenible en el tiempo. En ReservaUPLA, los cuatro factores señalados actuaron como un sistema de controles complementarios y no como actividades aisladas. Cada uno aportó una dimensión distinta de calidad, y juntos explican por qué el prototipo pudo crecer durante nueve iteraciones sin perder coherencia.

11.1. Estándares: previsibilidad y reducción de la carga cognitiva

Los estándares de nomenclatura, estructura y commits hicieron que el código fuera predecible. Un desarrollador que conoce la convención sabe dónde buscar la regla de negocio, cómo se llama la interfaz que la expone y qué capa debe modificarse. Esa previsibilidad reduce el tiempo de localización de defectos y disminuye el riesgo de introducir cambios en el lugar equivocado. En términos de la norma ISO/IEC 25010, los estándares inciden directamente en la **mantenibilidad** —modularidad, reusabilidad y facilidad de análisis— y en la **portabilidad**, porque la separación de capas permite sustituir la base de datos o la interfaz sin reescribir el dominio.

La arquitectura limpia es el ejemplo más claro. Las reglas institucionales de la UPLA viven en la capa de aplicación y no dependen de PostgreSQL ni de React. Si la universidad decidiera cambiar el motor de base de datos o habilitar un cliente móvil, las reglas RN-01 a RN-06 seguirían siendo las mismas. Esa independencia es lo que convierte un prototipo en una base evolutiva y no en una demostración desechable.

11.2. Validación: confianza en el dato y prevención del error

La validación en dos niveles cumple funciones distintas y complementarias. En el frontend mejora la **usabilidad** porque el usuario recibe retroalimentación inmediata, junto al campo, sin esperar una respuesta del servidor. En el backend protege la **integridad** y la **seguridad**, porque ninguna petición puede evadir las reglas aunque se construya fuera de la interfaz. Sin esta doble barrera, el sistema podría almacenar reservas solapadas, horarios fuera del rango institucional o decisiones tomadas por usuarios sin permiso.

La revalidación de RN-01 al aprobar ilustra por qué la validación no puede ser un trámite único. Dos solicitudes pendientes pueden convivir en el mismo horario; el conflicto aparece cuando una se aprueba. Detectar ese caso exige volver a consultar el estado en el momento de la decisión. Esta lógica convierte la validación en un control continuo y no en una formalidad de entrada, lo que se traduce en **fiabilidad** del servicio.

11.3. Reutilización: coherencia y economía de esfuerzo

El catálogo de componentes redujo la duplicación y, sobre todo, garantizó que la misma interacción se comporte igual en todo el sistema. Un aviso, un botón o un diálogo de confirmación no se reimplementan en cada página: se configuran. Esa uniformidad beneficia al usuario —que aprende una vez y reconoce en todas partes— y al equipo, que corrige un defecto en un solo lugar.

La reutilización también fue una decisión de accesibilidad. Implementar la trampa de foco, el foco visible y los anuncios `aria-live` en un componente de diálogo asegura que todos los diálogos del sistema hereden esas propiedades. Si se hubieran construido por separado, cada uno habría requerido su propia revisión y era probable que alguna quedara incompleta. En consecuencia, la reutilización no solo ahorra código: eleva el piso mínimo de calidad de toda la aplicación.

11.4. Refinamiento: reducción de deuda y capacidad de evolución

El refinamiento continuo permitió eliminar duplicaciones que, con el tiempo, se convierten en inconsistencias. La desaparición de la vista redundante de creación de reservas, la centralización del cliente HTTP, la unificación de las validaciones y la creación de constantes de dominio son intervenciones pequeñas que, acumuladas, reducen la complejidad accidental. El código resultante es más fácil de leer, de probar y de modificar.

Reconocer la deuda pendiente —pruebas automatizadas, paginación, límite de intentos y rotación de tokens— es parte del mismo ejercicio. Un sistema evoluciona mejor cuando sus carencias están identificadas y priorizadas que cuando se presentan como inexistentes. La documentación de esa deuda convierte el refinamiento en un proceso permanente y no en una tarea que se declara terminada.

11.5. Conclusión

En ReservaUPLA, los estándares aportaron orden; la validación, confianza en el dato; la reutilización, coherencia; y el refinamiento, capacidad de cambio. Ninguno de los cuatro habría bastado por separado: un sistema puede tener estándares y aun así carecer de validación; puede reutilizar componentes y, al mismo tiempo, acumular deuda si no se refina. La calidad surge de la interacción entre ellos y se manifiesta en tres resultados concretos: un usuario que comprende qué hacer y recibe mensajes útiles, una institución que puede confiar en el registro de ocupación y un equipo que puede continuar el desarrollo sin reconstruir lo existente. Esa combinación es, en definitiva, lo que distingue un prototipo funcional de un producto de software sostenible.

REFERENCIAS BIBLIOGRÁFICAS

- Fowler, M. (2018). Refactoring: Improving the design of existing code (2.ª ed.). Addison-Wesley.
- International Organization for Standardization. (2010). ISO 690:2010. Information and documentation — Guidelines for bibliographic references and citations to information resources.
- International Organization for Standardization. (2011). ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
- International Organization for Standardization. (2018). ISO 9241-11:2018. Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts.
- International Organization for Standardization. (2019). ISO 9241-210:2019. Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems.
- ISO/IEC/IEEE. (2017). ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes.
- Martin, R. C. (2018). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- McConnell, S. (2004). Code complete: A practical handbook of software construction (2.ª ed.). Microsoft Press.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3.ª ed.). John Wiley & Sons.
- Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.
- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9.ª ed.). McGraw-Hill Education.
- Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). Designing the user interface: Strategies for effective human-computer interaction (6.ª ed.). Pearson.
- Sommerville, I. (2019). Ingeniería de software (10.ª ed.). Pearson Educación.

ANEXO A. ESPECIFICACIÓN TÉCNICA DE LA API BACKEND (OPENAPI / SWAGGER)

Este anexo documenta el contrato de la API backend expuesta mediante OpenAPI. La tabla siguiente enumera los endpoints implementados con su método, ruta, función y nivel de acceso; a continuación se presenta la vista interactiva de Swagger UI, que publica el cuerpo de la solicitud y las respuestas 201, 400 y 409 del registro de reservas. Esta es la única evidencia de infraestructura que se conserva en los anexos: las capturas de la interfaz se analizan en los capítulos de cada semana.

Tabla 30. Endpoints de la API y autorización requerida *Fuente: elaboración propia.*

Método	Ruta	Función	Acceso
POST	/api/auth/login	Autenticación y emisión de JWT	Público
GET	/api/auth/perfil	Datos del usuario autenticado	Autenticado
GET	/api/usuarios	Listado de usuarios	Administrador
POST	/api/usuarios	Alta de usuario	Administrador
PUT	/api/usuarios/{id}	Actualización de usuario	Administrador
PATCH	/api/usuarios/{id}/estado	Activación o suspensión	Administrador
DELETE	/api/usuarios/{id}	Baja o desactivación	Administrador
GET	/api/ambientes	Catálogo de ambientes	Autenticado
POST	/api/ambientes	Alta de ambiente	Administrador
PUT	/api/ambientes/{id}	Actualización de ambiente	Administrador
PATCH	/api/ambientes/{id}/estado	Cambio de estado operativo	Administrador
DELETE	/api/ambientes/{id}	Baja o desactivación	Administrador
GET	/api/reservas	Listado con filtros	Autenticado
GET	/api/reservas/{id}	Detalle de la reserva	Autenticado
POST	/api/reservas	Registro de solicitud	Solicitante y Administrador
PATCH	/api/reservas/{id}/estado	Aprobación, rechazo o cancelación	Administrador o solicitante
GET	/api/dashboard	Indicadores consolidados	Autenticado
GET	/api/health	Verificación de disponibilidad	Público

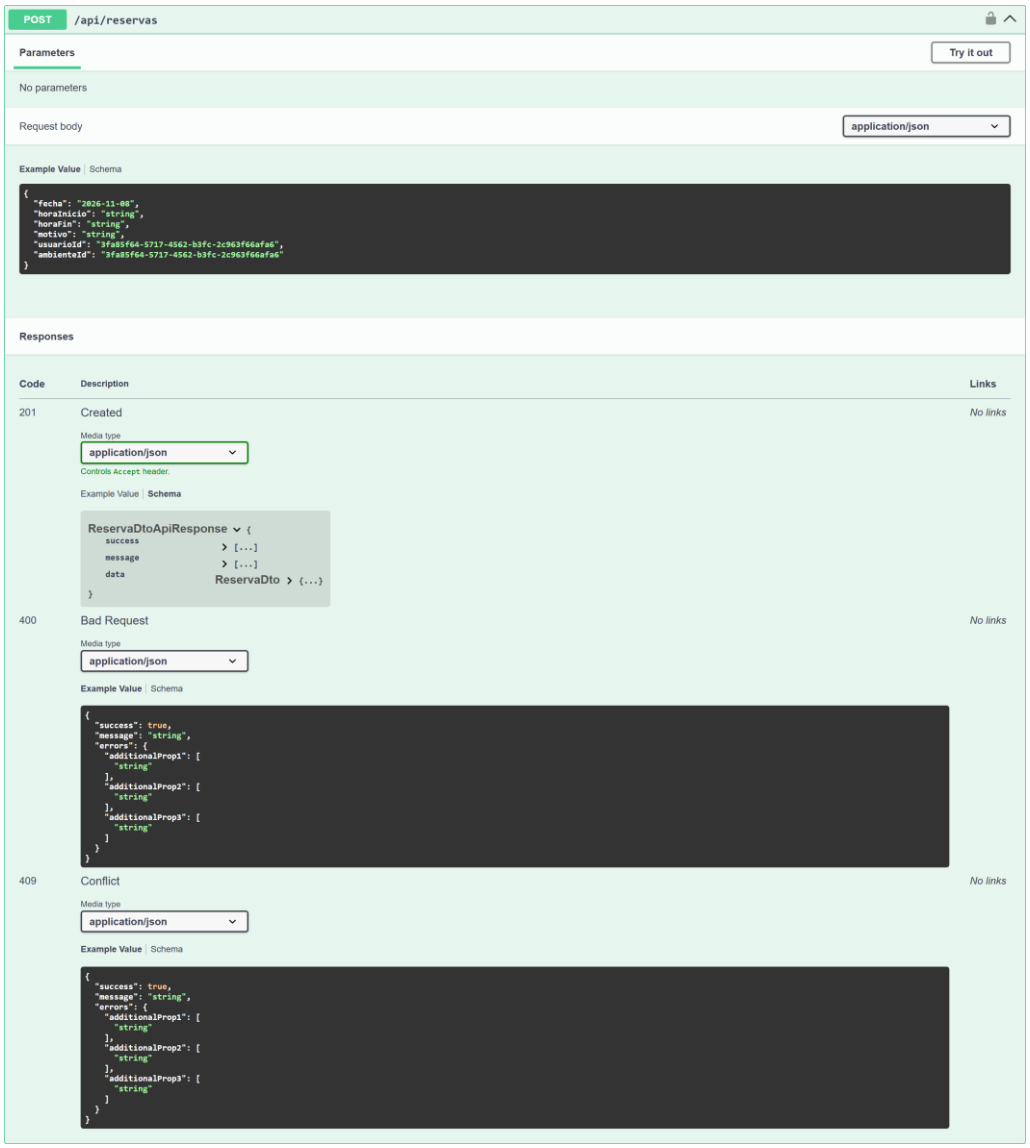


Figura 38. Contrato OpenAPI del endpoint de registro de reservas en Swagger UI

El contrato declara de forma explícita las respuestas 201, 400 y 409. La existencia del código 409 no es un detalle de implementación: expresa que el solapamiento de horarios es un resultado previsto del dominio y que el cliente debe tratarlo como un conflicto de negocio, no como una falla técnica.